



Ministerul Educației și Cercetării
Universitatea "OVIDIUS" Constanța
Facultatea de Matematică și Informatică
Specializarea Informatică

Aplicație software de pontare a participanților la evenimente

Lucrare de licență

Coordonator științific:

Conf. univ. dr. Puchianu Crenguța-Mădălina

Absolvent:
Moraru Vlad

Constanța
2026

Cuprins

Cuprins	i
Lista Figurilor	iv
1 Introducere	1
1.1 Structura lucrării	2
2 Definirea problemei	3
2.1 Introducere și Contextualizare	3
2.2 Analiza soluțiilor existente	4
2.2.1 Sistemele cu cartele magnetice	4
2.2.2 Sisteme mobile bazate pe scanare de coduri QR	4
2.2.3 Sisteme de tracking pe bază de localizare GPS	5
2.2.4 Sinteza comparativă	6
2.2.5 Fundamentarea arhitecturală a soluției PresSync	6
2.3 Descrierea problemei și soluția propusă	7
2.4 Obiectivele sistemului	8

2.5	Actori principali și delimitarea rolurilor	8
2.6	Scenarii de bază	9
3	Analiza Software a Sistemului	10
3.1	Documentul de cerințe	10
3.1.1	Actori și roluri în raport cu sistemul PresSync	11
3.1.2	Cerințe funcționale	11
3.1.3	Cerințe nefuncționale	12
3.2	Cazuri de utilizare	13
3.2.1	Creare Cont	13
3.2.2	Vizualizare Prezență cu Statistică	14
3.2.3	Înregistrare Prezență	15
3.2.4	Administrează Prezență Manual	15
3.2.5	Creare Categori de Evenimente	16
3.2.6	Creare Evenimente	16
3.3	Diagrame de secvențe de sistem	17
3.3.1	Evenimente de sistem	17
3.3.2	Diagrame de secvență de sistem	19
3.4	Modelul domeniului	24
4	Proiectarea și implementarea aplicației	26
4.1	Arhitecturi și stiluri arhitecturale	26
4.2	Modele de proiectare folosite de atribuire a responsabilităților	27
4.3	Diagrame de interacțiune	29
4.3.1	Creare Cont	29

4.3.2	Creare Evenimente	30
4.3.3	Înregistrare Prezență	40
4.3.4	Vizualizare prezență cu statistici	42
4.4	Diagrame de clase de proiectare	47
4.5	Modelarea bazei de date	48
5	Concluzii	50
6	Anexe	51
	Referințe bibliografice	65

Lista Figurilor

3.1	Tipuri de relații în diagrama UML de cazuri de utilizare	17
3.2	Diagrama cazurilor de utilizare	18
3.3	Prezentarea diagramelor de secvență de sistem	20
3.4	Diagrama de secvență — Creare cont	20
3.5	Diagrama de secvență — Creare evenimente	21
3.6	Diagrama de secvență — Înregistrare prezentă	21
3.7	Diagrama de secvență — Vizualizare prezentă	22
3.8	Diagrama de context	23
3.9	Explicarea relațiilor în modelul domeniului	24
3.10	Diagrama de clase a modelului domeniului	25
4.1	Diagrama de interacțiune — Creare cont	29
4.2	Implementare AuthenticationController	29
4.3	Implementare AuthenticationService	30
4.4	Interfața de înregistrare — formularul gol	31
4.5	Interfața de înregistrare — eroare nume invalid	32

4.6	Interfața de înregistrare — eroare email invalid	33
4.7	Interfața de înregistrare — eroare parolă prea scurtă	34
4.8	Diagrama de interacțiune — Creare evenimente	34
4.9	Implementare EventCategoryController	35
4.10	Implementare CreateEventCategoryCommand	35
4.11	Interfața de creare categorii — butonul de creare	36
4.12	Interfața de creare categorii — panoul de creare	36
4.13	Interfața de creare categorii — eroare 1	37
4.14	Interfața de creare categorii — eroare 2	38
4.15	Interfața de creare categorii — eroare 3	39
4.16	Diagrama de interacțiune — Înregistrare prezență	40
4.17	Implementare AttendanceController	40
4.18	Implementare CreateAttendanceCommand	41
4.19	Diagrama de interacțiune — Vizualizare prezență cu statistici	42
4.20	Vizualizare prezență — Admin	43
4.21	Vizualizare prezență — Săptămânal	44
4.22	Vizualizare prezență — Zilnic	45
4.23	Implementare EventCategoryController — getCategory	45
4.24	Implementare GetEventStatsQuery	46
4.25	Diagrama de clase de proiectare — Creare cont	47
4.26	Diagrama de clase de proiectare — Înscriere prezență	47
4.27	Structura bazei de date	48
6.1	Diagrama de activitate — Creare cont	51

6.2	Diagrama de activitate — Autentificare	52
6.3	Diagrama de activitate — Administrare evenimente	53
6.4	Diagrama de activitate — Administrare evenimente repetitive	54
6.5	Diagrama de activitate — Arhivare eveniment	55
6.6	Diagrama de activitate — Creare categorii	56
6.7	Diagrama de activitate — Editare categorii	57
6.8	Diagrama de activitate — Editare evenimente	58
6.9	Diagrama de activitate — Administrare utilizator	59
6.10	Diagrama de activitate — Căutare utilizator	60
6.11	Diagrama de activitate — Vizualizare stare utilizator	61
6.12	Diagrama de activitate — Administrare prezență manuală	62
6.13	Diagrama de activitate — Înregistrare prezență	63
6.14	Diagrama de activitate — Vizualizare prezență cu statistică	64

Capitolul 1

Introducere

Prezența participanților la activități educaționale, profesionale sau organizaționale este o provocare semnificativă în contextul actual, în care fiabilitatea datelor și eficiența operațională sunt esențiale. Metodele tradiționale de pontaj — apelul nominal, listele pe hârtie sau cartelele de acces fizice — sunt lente, predispuse la erori și vulnerabile la fraudă, precum înregistrarea prezenței în numele unui coleg absent. Soluțiile digitale existente, bazate pe coduri QR, localizare GPS sau platforme cloud, adresează parțial aceste neajunsuri, însă introduc alte limitări: dependența de conexiunea la internet, riscul falsificării locației sau preocupări legate de confidențialitatea datelor personale.

Lucrarea de față propune ca soluție PresSync — un sistem autonom de gestionare a prezenței, care utilizează infrastructura Wi-Fi locală ca mecanism principal de validare a prezenței fizice. Principiul de funcționare se bazează pe conectivitatea la nivelul Stratului 2 (Data Link Layer) al modelului OSI [1]: un router configurat în regim de portal captiv emite pe benzile de 2,4 GHz și 5 GHz, iar un dispozitiv mobil se poate conecta doar dacă se află fizic în raza de acoperire a semnalului. Pereții și obstacolele din mediul înconjurător atenuează natural semnalul Wi-Fi, transformând incinta în care are loc evenimentul într-o barieră fizică antifraudă.

Sistemul este găzduit integral pe un router OpenWRT și nu necesită conexiune la internet sau servere externe, funcționând într-un mediu de rețea complet izolat. Arhitectura software se bazează pe principiul Command-Query Separation (CQS) și este implementată în Java cu ajutorul framework-ului Spring, iar interfața cu utilizatorul este o aplicație web accesibilă prin portalul captiv al routerului. Autentificarea utilizatorilor se realizează prin token-uri JWT, iar separarea strictă a rolurilor (administrator, moderator, participant) asigură respectarea principiului privilegiului minim.

Prin această abordare, PresSync oferă o soluție accesibilă din punct de vedere economic, ușor de implementat în orice spațiu cu alimentare electrică și care garantează, prin proprietățile fizice ale undelor radio, că participanții înregistrați sunt prezenți în mod real la eveniment.

1.1 Structura lucrării

Lucrarea este structurată în cinci capitole, organizate progresiv de la analiza problemei până la implementarea și evaluarea soluției propuse.

- ▷ **Capitolul 1 — Introducere** oferă o imagine de ansamblu asupra contextului care a motivat dezvoltarea sistemului PresSync, prezintă pe scurt soluția propusă și descrie structura lucrării.
- ▷ **Capitolul 2 — Definirea problemei** analizează metodele actuale de pontaj și limitările acestora, realizează o comparație structurată a soluțiilor existente pe piață și fundamentează arhitectural alegerea abordării bazate pe conectivitate Wi-Fi la nivelul Stratului 2. De asemenea, capitolul definește obiectivele sistemului, actorii implicați și scenariile de bază de utilizare.
- ▷ **Capitolul 3 — Analiza software** prezintă cerințele funcționale și non-funcționale ale aplicației, modelul funcțional al sistemului, descrierea detaliată a cazurilor de utilizare și diagramele UML asociate.
- ▷ **Capitolul 4 — Proiectare și implementare** descrie arhitectura tehnică a soluției, incluzând proiectarea bazei de date, tehnologiile utilizate, diagramele de interacțiune și de clase, precum și secvențe reprezentative de cod sursă.
- ▷ **Capitolul 5 — Concluzii** sintetizează contribuțiile proprii aduse prin această lucrare, evaluează rezultatele obținute în raport cu obiectivele definite și propune direcții de dezvoltare ulterioară a sistemului.

Capitolul 2

Definirea problemei

Acest capitol definește problema gestionării prezenței în diferite contexte și oferă o imagine de ansamblu asupra sistemelor actuale de pontaj și a limitărilor acestora. De asemenea, este realizată o analiză comparativă a modelelor existente cu soluția propusă de mine și se enumeră obiectivele acesteia. Capitolul se încheie cu definirea actorilor, descrierea sistemului și prezentarea scenariilor de bază de utilizare.

2.1 Introducere și Contextualizare

Efectuarea prezenței participanților la cursuri, ședințe sau conferințe este, de multe ori, o activitate plictisitoare și consumatoare de timp atât pentru cel care realizează această activitate, cât și pentru participanți. Listele pe hârtie circulate prin sală, strigarea fiecărui nume sau distribuirea cartelelor de acces sunt soluții banale, dar ele ascund probleme reale: timp pierdut, erori frecvente și participanți care sunt pe listă, deși nu se află în sală.

Această situație, unde un coleg îl marchează pe altul prezent, deși nu e adevărat, nu este doar o nedreptate față de ceilalți, dar creează și o greșală în datele colectate. Acest lucru poate afecta decizii ce țin de evaluare sau de monitorizarea activității. Această problemă nu este una trivială, deoarece, cu cât grupul este mai mare, cu atât crește riscul ca astfel de situații să treacă neobservate.

Este important să evidențiem că această problemă nu aparține doar mediului universitar, chiar dacă soluția mea a pornit de acolo. Ea poate fi aplicată și în medii cum ar fi o conferință sau orice alt tip de eveniment unde organizatorul ar avea nevoie să știe exact numărul de participanți și cine sunt ei, pentru a crea mai apoi statistici sau rapoarte pentru planificarea următorului eveniment. Deci, cum verificăm că oamenii sunt cu adevărat acolo, fără a întrerupe activitatea și fără a permite date inexacte? Avem nevoie de un mecanism de înregistrare a prezenței care să fie fiabil, rapid și capabil să genereze informațiile utile, nu o simplă listă bifată în grabă.

2.2 Analiza soluțiilor existente

Pe piață există deja o varietate largă de soluții care gestionează prezența, de la liste de hârtie și carduri fizice până la platforme cloud sofisticate. Problema nu este lipsa soluțiilor, ci faptul că multe dintre ele nu rezolvă în totalitate problema.

Unele sisteme necesită costuri suplimentare pentru a fi implementate, necesitând echipamente hardware dedicate și o infrastructură tehnică. Alte soluții sunt mai accesibile, dar nu oferă garanția că persoana se află fizic în încăpere.

În toate mediile, nevoia principală rămâne aceeași: o soluție ușor de folosit, de implementat și care nu permite fraudă. Acest echilibru este ceea ce lipsește majorității soluțiilor. Pentru a înțelege exact unde se situează PresSync, urmează să prezint o comparație cu cele trei tipuri de sisteme care sunt cel mai des utilizate în prezent.

2.2.1 Sistemele cu cartele magnetice

Cartelele magnetice sunt probabil cele mai cunoscute soluții de control al accesului pe care le putem întâlni. Funcționarea lor se bazează pe principiul că participantul deține un obiect fizic, pe care îl prezintă unui cititor la intrarea în spațiu. Prezența și autentificarea au loc instant. Simplitatea este punctul forte al acestui tip de soluții: scanarea rapidă, infrastructura conectată prin cablu la serverele instituției și funcționarea într-o rețea izolată.

Problema principală o constituie costurile, care sunt semnificative — cumpărarea și instalarea cititoarelor la fiecare intrare, mentenanța continuă și înlocuirea periodică a cartelelor pierdute sau deteriorate. De asemenea, sistemul este ancorat de acea zonă. Mai mult de atât, există și riscul ca un coleg să îi dea cartela altuia pentru a fi marcat prezent.

2.2.2 Sisteme mobile bazate pe scanare de coduri QR

Codul QR afișat pe un videoproiector a devenit ceva familiar în sălile de curs în ultimii ani. Se generează un cod QR, care mai apoi este proiectat, iar studenții scanează codul cu telefonul. Implementarea nu necesită hardware suplimentar, sistemul se folosește de tehnologia existentă deja și poate fi pus în funcțiune în câteva minute.

Această abordare este una ideală în contextul educațional, unde bugetul este limitat. Însă problema acestei soluții este securitatea, deoarece ea se bazează pe faptul că doar persoanele

prezente fizic au acces la cod, însă această ipoteză nu poate fi adevărată. Un singur participant din sală poate fotografia codul și îl trimite pe WhatsApp oricui. Nici versiunile cu coduri dinamice nu rezolvă complet problema: codul poate fi partajat în timp real.

2.2.3 Sisteme de tracking pe bază de localizare GPS

Aceste platforme au o abordare diferită. Ele verifică dacă participantul se află în locul potrivit. Utilizatorul are nevoie doar să facă un check-in din aplicație, iar sistemul compară coordonatele GPS ale telefonului cu o zonă predefinită în aplicație.

Această soluție este flexibilă, nu are nevoie de o infrastructură hardware și evenimentul poate fi configurat oriunde. Totuși, pentru săli de curs, birouri etc., această abordare are trei probleme.

Prima este falsificarea coordonatelor GPS. Există aplicații specializate care permit simularea unei locații. A doua problemă este confidențialitatea. Și a treia limitare este de natură tehnică, sistemul are nevoie de o conexiune constantă la internet. În spații cu semnal slab, aplicația devine inoperabilă.

2.2.4 Sinteza comparativă

Criteriu comparativ	Sisteme RFID / Cartele	Scanare Cod QR	Tracking GPS (Cloud)	PresSync
Costuri de implementare	Ridicate (necesită hardware dedicat și cartele)	Foarte scăzute (folosește infrastructura mobilă existentă)	Medii spre ridicate (abonamente SaaS/Cloud)	Scăzute (necesită un router compatibil OpenWRT și/sau un sistem embedded)
Risc de fraudă	Ridicat (delegarea fizică a token-ului de autentificare)	Foarte ridicat (redistribuirea codului optic în afara perimetrului)	Ridicat (falsificarea coordonatelor GPS prin location mocking)	Scăzut (conectivitatea la nivelul Stratului 2 este condiționată de atenuarea fizică a semnalului Wi-Fi)
Versatilitate	Redusă (infrastructură fixă, instalată permanent)	Ridică	Foarte ridicată	Ridică (echipament portabil, instalabil în orice spațiu cu alimentare electrică)
Dependență de conexiune externă	Nu (funcționează de regulă în rețea Intranet)	Da	Da (obligatoriu)	Nu (sistemul funcționează într-un mediu de rețea complet izolat, fără conexiune externă)

Tabelul 2.1: Comparatie între principalele soluții de gestionare a prezenței

2.2.5 Fundamentarea arhitecturală a soluției PresSync

Analizând comparația, putem să ne dăm seama că fiecare abordare expune o problemă, fie securitatea, fie accesibilitatea economică. PresSync a fost conceput pentru a rezolva toate aceste probleme prin valorificarea proprietății fizice a semnalului Wi-Fi.

Primul pas este înlocuirea cartelei cu dispozitivul mobil personal, ceea ce practic rezolvă împrumutarea token-ului unui coleg. Identitatea digitală a participantului este stocată sub forma unui token JWT, legat de contul său și accesibil exclusiv prin autentificare, ceea ce transformă dispozitivul într-un factor de autentificare.

Elementul principal al arhitecturii PresSync, și diferența cea mai mare față de soluțiile analizate mai sus, este folosirea proprietăților fizice ale undelor radio ca barieră antifraudă. Această abordare este mai robustă, deoarece validarea prezenței se face prin conectivitate la nivelul Stratului 2 (Data link layer) al modelului OSI [1]. Routerul este configurat în

regim de portal captiv și emite pe benzile 2,4 GHz și 5 GHz, frecvențe care sunt sensibile la obstacole fizice din mediul înconjurător.

Atunci când traversează materiale de construcție uzuale, cum ar fi pereți din beton, cărămidă sau gips-carton, semnalul suferă o atenuare destul de mare, ceea ce face ca semnalul emis să fie insuficient pentru ca un dispozitiv să se conecteze de la câțiva metri de punctul de acces. Prin urmare, pentru a transmite token-ul JWT și a înregistra prezența, telefonul trebuie să se afle fizic în interiorul spațiului delimitat de router.

Încă un factor care contribuie la această restricție naturală este zona Fresnel. Obstacolele solide intersectate de această zonă reduc puterea semnalului. Combinând acești doi factori, transformăm pereții încăperii într-o barieră naturală antifraudă.

Acest ultim factor previne atacurile de tip MAC spoofing care necesită prezența fizică în raza de acțiune a routerului și echipament specializat.

Încă un factor important al sistemului este independența față de conexiunea la internet. Aplicația rulează local, nu există servere externe, abonamente cloud etc. Această alegere aduce două consecințe practice: prima este că sistemul funcționează în orice spațiu cu alimentare electrică. A doua este că datele sunt procesate local fără a fi transmise la alte servicii.

În concluzie, PresSync rezolvă problema la un nivel mai profund, ancorând validarea prezenței în proprietăți fizice și eliminând dependența de infrastructura externă.

2.3 Descrierea problemei și soluția propusă

Pe baza secțiunilor prezentate mai sus, putem formula problema principală pe care sistemul o abordează: necesitatea unui mecanism de înregistrare a prezenței care să ofere certitudinea prezenței fizice reale a participanților, fără prea multe costuri sau dependențe de conexiunea externă și păstrarea datelor local. Având în vedere deficiențele soluțiilor deja existente menționate anterior, am considerat necesară construirea unei soluții găzduite local pe un router OpenWRT, în care prezența fizică în rețea este dovada participării, ceea ce elimină dependența de internet sau servicii GPS. PresSync automatizează pontajul, printr-o metodă sigură, rapidă și utilă. Contextele de aplicare pot fi variate, de la mediul academic până la conferințe și evenimente temporare.

2.4 Obiectivele sistemului

Pentru a materializa viziunea descrisă mai sus, am fost ghidat de următoarele obiective funcționale și tehnice:

- O1. Funcționare autonomă și independență hardware.** Sistemul este proiectat să ruleze izolat pe echipamente accesibile, precum un router OpenWRT. Această abordare elimină complet dependența de o conexiune la internet sau de servere externe, asigurând un mediu securizat în care prezența în rețeaua fizică constituie dovada fundamentală a participării.
- O2. Automatizarea calendarului și a recurenței.** Sistemul preia munca repetitivă a administratorului. Odată definită o categorie de evenimente împreună cu regulile sale de recurență (zilnic, săptămânal etc.), aplicația instanțiază automat noi evenimente la momentul oportun, fără nicio intervenție manuală.
- O3. Securizarea și delimitarea strictă a accesului.** Autentificarea utilizatorilor este realizată prin *token*-uri JWT, asigurând un flux clar de autorizare. Fiecare actor are vizibilitate limitată la resursele relevante: participantul își confirmă prezența și își verifică istoricul personal, în timp ce organizatorul administrează resursele colective și accesează datele întregului grup.
- O4. Procesare concurentă și scalabilitate ridicată.** Sistemul separă operațiile de modificare a datelor (comenzi) de cele de citire. Această separare asigură că cererile de marcarea a prezenței sunt procesate instantaneu și fără conflicte, iar cererile complexe de calcul al statisticilor rulează pe un flux propriu optimizat, fără a bloca aplicația.
- O5. Generarea de statistici în timp real.** În loc să stocheze datele în formă brută, sistemul le transformă în informații utile la cerere. Acesta calculează procentajul de prezență la nivel de utilizator sau eveniment și permite extragerea de rapoarte detaliate direct din interfață.

2.5 Actori principali și delimitarea rolurilor

Sistemul PresSync lucrează cu două categorii principale de utilizatori, pe care le putem considera *stakeholder*-ii primari și secundari ai platformei.

Administratorul (Organizatorul) reprezintă *stakeholder*-ul primar. El este responsabil de configurarea evenimentelor și a categoriilor acestora, de gestionarea participanților, de stabilirea regulilor de recurență (atunci când evenimentele se repetă) și de generarea rapoartelor statistice. Acest rol are acces complet la toate funcționalitățile de administrare ale sistemului.

Participantul este *stakeholder*-ul secundar. El interacționează cu platforma doar pentru a-și

marca prezența la evenimentele la care este înscris și pentru a consulta propriul istoric de participare. Accesul său este strict limitat la informațiile și acțiunile care îl privesc direct.

Această separare clară între roluri nu este întâmplătoare. Ea asigură atât o protecție mai bună a datelor, cât și o experiență adaptată nevoilor fiecărui tip de utilizator. Totodată, respectă principiul privilegiului minim (*principle of least privilege*) [2], conform căruia un utilizator ar trebui să aibă doar accesul strict necesar pentru a-și îndeplini atribuțiile.

2.6 Scenarii de bază

Pentru o mai bună înțelegere a sistemului voi explica pe pași cum are loc procesul de marcare a prezenței, creare evenimente, vizualizare prezențe și administrare utilizatori.

Pentru acest exemplu vom folosi mediul academic.

- ▷ Profesorul Vlad Moraru, având deja sistemul activ și fiind conectat la rețea, deschide o pagină web și este redirecționat de către portalul captiv pe pagina aplicației PresSync. Se înregistrează sau se autentifică dacă are deja cont, iar, fiind moderator/administrator, are nevoie de un cod unic care este transmis pe email-ul introdus.
- ▷ După completarea formularului este redirecționat pe pagina principală și la apăsarea butonului „Creează categorie de eveniment”, este afișat un formular unde introduce datele despre categoria de eveniment pe care vrea să o creeze.
- ▷ Sistemul folosește categoria de eveniment ca un template și apoi creează un eveniment la data și ora setată.
- ▷ Studentul, aflat în sala de curs și fiind conectat la rețeaua oferită de router, deschide browserul și este redirecționat către pagina de autentificare, completează formularul și, dacă există un eveniment activ și prezența este deschisă, sistemul îl pune ca prezent, fără să apese niciun buton în plus.
- ▷ Pentru a vizualiza prezența, ambii au posibilitatea de a căuta categoria de eveniment dorită și pentru fiecare informațiile vor fi diferite: pentru student va fi afișat doar propriile prezențe, iar pentru ceilalți actori vor fi afișate prezențele tuturor participanților.

Capitolul 3

Analiza Software a Sistemului

În timpul analizei software a sistemului creăm modele conceptuale, care ne ajută să abstractizăm domeniul problemei și să-l înțelegem mai ușor. În plus, cu ajutorul acestor modele, putem să identificăm riscuri sau neajunsuri ale unui sistem și să ne dăm seama despre cum ar trebui să arate soluția în finalitate. Aici voi prezenta documentul de cerințe, diagrama UML de clase a modelului domeniului și diagrame UML de secvențe de sistem. În documentul de cerințe prezentăm actorii care participă la funcționarea sistemului și modurile în care sistemul poate fi folosit. Diagrama UML de secvențe de sistem ne ilustrează modul în care actorii comunică prin mesaje cu sistemul, mesajele și ordinea lor.

3.1 Documentul de cerințe

Platforma PresSync a fost gândită ca o soluție completă pentru monitorizarea în timp real a prezenței la diverse activități și evenimente. Ea integrează mai multe module care lucrează împreună, urmărind atât fluxurile utilizatorilor, cât și managementul evenimentelor. În esență, sistemul centralizează tot ce ține de conturi, configurarea activităților, înscrierea participanților și validarea prezenței, combinând automatizări cu posibilitatea de intervenție manuală acolo unde este necesar.

3.1.1 Actori și roluri în raport cu sistemul PresSync

Admin – Are control total asupra platformei. El poate gestiona conturile de Moderator, configurează permisiunile globale de securitate și are acces la toate setările critice ale sistemului.

Moderator

– Se ocupă de partea operațională zilnică. El creează și gestionează evenimentele, coordonează utilizatorii înscriși și poate genera rapoarte de prezență sau audit.

User – Reprezintă participantul obișnuit. Poate să se înscrie la evenimente, să-și vadă istoricul personal de participare și să acceseze informațiile relevante pentru el.

3.1.2 Cerințe funcționale

F1. Gestiunea fluxurilor de înregistrare și autentificare

F1.1 Utilizatorii se pot înregistra în sistem prin formulare dedicate, în funcție de rolul ales (Admin, Moderator sau User).

F1.2 Datele introduse sunt verificate atât din punct de vedere sintactic, cât și semantic. Dacă apar erori, utilizatorul vede alerte clare, dar informațiile completate corect nu se pierd.

F1.3 După finalizarea cu succes a înregistrării, sistemul creează automat contul și redirecționează utilizatorul către pagina de profil.

F1.4 Autentificarea se face prin nume de utilizator și parolă. În funcție de rol, interfața se ajustează dinamic, afișând doar funcționalitățile permise.

F1.5 Dacă utilizatorul renunță la procesul de login sau înregistrare, este redirecționat automat către pagina principală.

F2. Managementul ciclului de viață al evenimentelor

F2.1 Un eveniment nou poate fi creat prin completarea unor informații esențiale: titlu, perioadă, locație, descriere și tipul activității.

F2.2 Moderatorii pot modifica detaliile unui eveniment oricând, iar ștergerea definitivă se face doar după o confirmare explicită, pentru a evita greșeli.

F2.3 Există o listă centralizată a tuturor evenimentelor, cu posibilitatea de a vizualiza detaliile complete ale fiecăruia printr-un simplu click.

F3. Administrarea conturilor de utilizator

- F3.1** Crearea și configurarea conturilor de Admin este permisă doar utilizatorului cu drepturi maxime (Super Admin).
- F3.2** Sistemul permite blocarea temporară sau reactivarea conturilor atunci când este necesar (de exemplu, în caz de inactivitate îndelungată sau probleme de securitate).
- F3.3** Atât Adminul cât și Moderatorii au acces la o secțiune centralizată unde pot vedea listă completă a utilizatorilor și detaliile profilurilor acestora.

F4. Validarea și monitorizarea prezenței

- F4.1** Pentru a marca prezența, trebuie mai întâi selectată o activitate din catalog.
- F4.2** Prezența se poate înregistra atât manual de către organizator, cât și automatizat (unde este posibil), evitându-se astfel înregistrările duble pentru același eveniment.
- F4.3** În orice moment se poate vedea o situație centralizată, clar separată între participanții prezenți și cei absenți.

F5. Module de raportare și analiză

- F5.1** Sistemul agregă datele de prezență și generează diverse statistici utile privind gradul de participare la activități.
- F5.2** Rapoartele pot fi exportate atât în format PDF, cât și CSV, pentru a putea fi prelucrate ulterior.
- F5.3** Utilizatorii simpli au acces la propriul istoric de participări, afișat cronologic.

3.1.3 Cerințe nefuncționale

- CN1. Ergonomie și usabilitate** – Interfața trebuie să fie intuitivă și modernă, astfel încât un utilizator nou să se poată obișnui rapid cu platforma, indiferent de nivelul tehnic.
- CN2. Toleranță la erori și robustețe** – Aplicația va trata cu atenție erorile, oferind mesaje clare și prietenoase, fără termeni tehnici care să confuzeze utilizatorul.
- CN3. Mediu de dezvoltare și tehnologii** – Partea de backend va fi dezvoltată în Java folosind o arhitectură stratificată, ușor de întreținut și extins.
- CN4. Portabilitate** – Soluția trebuie să ruleze fără probleme pe Windows, Linux și macOS.
- CN5. Securitate și confidențialitate** – Parolele vor fi stocate hashed cu algoritmi puternici, iar accesul la funcționalități se va face prin RBAC (control granular pe roluri).

3.2 Cazuri de utilizare

Cazurile de utilizare reprezintă o tehnică de modelare utilizată pentru a descrie interacțiunile dintre actori și sistem în vederea atingerii unor obiective specifice. Fiecare caz de utilizare definește un set de scenarii care ilustrează modul în care sistemul răspunde la acțiunile actorilor, evidențiind atât fluxurile principale de succes, cât și fluxurile alternative sau excepțiile. Această abordare facilitează înțelegerea cerințelor funcționale și contribuie la identificarea limitelor sistemului. În continuare, prezentăm cazurile de utilizare software ale sistemului PresSync.

3.2.1 Creare Cont

Descriere: Procesul complet de înregistrare a unui utilizator nou în sistem. Utilizatorul completează datele personale, iar sistemul creează contul, criptează parola și trimite un cod OTP pe email pentru verificare înainte ca acesta să poată fi folosit.

Actor: Utilizator (principal)

Eveniment declanșator: Utilizatorul accesează pagina de înregistrare și trimite formularul cu datele personale.

Precondiții: Sistemul este operațional. Adresa de email nu este deja asociată unui cont existent.

Postcondiții: Contul este creat în baza de date (inițial inactiv), parola este stocată securizat folosind BCrypt, iar un cod OTP este trimis pe email pentru activare.

Flux principal:

Utilizator	Sistem
1. Cere crearea unui cont	2. Afișează formularul de înregistrare
3. Completează datele	4. Verifică datele primite (sintactic și semantic) [A1][A2]
	5. Creează cont nou și stochează parola securizat
	6. Memorează datele contului creat
	7. Afișează mesaj de succes

Fluxuri alternative:

[A1] **Datele sunt introduse eronat** Sistemul afișează un mesaj de eroare clar și marchează câmpurile greșite. Utilizatorul poate corecta informațiile fără a pierde ce a completat deja.

[A2] **Utilizatorul există deja (eroare semantică):** Sistemul detectează că username-ul sau email-ul este deja folosit, afișează mesajul „Contul există deja” și revine la formular.

3.2.2 Vizualizare Prezență cu Statistică

Descriere: Utilizatorul vizualizează istoricul propriu de prezență și primește statistici personale (inclusiv predicții). Adminul poate vizualiza statistici agregate, aplica filtre și genera rapoarte detaliate.

Actor: Utilizator / Admin (principal)

Eveniment declanșator: Actorul selectează opțiunea „Statistici” sau „Rapoarte” din meniul principal.

Precondiții: Actorul este autentificat și există date de prezență înregistrate.

Postcondiții: Sistemul afișează datele solicitate sub formă de tabel și/sau grafice.

Flux principal (Admin):

Admin	Sistem
1. Selectează „Rapoarte și Statistici”	2. Afișează dashboard-ul cu statistici agregate
3. Aplică un filtru (perioadă, eveniment, grupă etc.)	4. Validează criteriile de filtrare [A1]
	5. Afișează listă de rezultate
6. Selectează un utilizator sau eveniment	7. Generează raportul detaliat
8. Selectează „Descarcă Raport”	9. Generează fișierul (PDF/CSV) [A2]
	10. Afișează mesaj de succes

Flux principal (Utilizator):

Utilizator	Sistem
1. Selectează „Statistici”	2. Aplică automat filtrul pe ID-ul propriu
	3. Generează și afișează istoricul de prezență
4. Selectează un eveniment specific	5. Afișează detaliile evenimentului

Fluxuri alternative:

[A1] **Criterii de filtrare invalide:** Sistemul afișează eroare și marchează câmpurile greșite.

[A2] **Eșec la generarea raportului:** Sistemul afișează un mesaj clar de eroare tehnică.

[A3] **Nu există date:** Sistemul afișează mesajul „Nu au fost găsite date de prezență înregistrate.”

3.2.3 Înregistrare Prezență

Descriere: Procesul prin care un utilizator își înregistrează prezența la un eveniment activ. Sistemul verifică existența evenimentului și dreptul de acces al utilizatorului.

Actor: Utilizator (principal)

Eveniment declanșator: Utilizatorul accesează funcția de prezență sau se autentifică în timpul unui eveniment activ.

Precondiții: Există un eveniment activ la care utilizatorul nu are deja prezența înregistrată și nu este restricționat.

Postcondiții: Sistemul salvează prezența utilizatorului la evenimentul activ.

Flux principal:

Utilizator	Sistem
1. Cere înregistrarea prezenței	2. Verifică evenimente active și drepturi [A1] 3. Afișează detaliile evenimentului curent
4. Confirmă prezența [A2]	5. Salvează prezența 6. Afișează mesaj de înregistrare completă

Fluxuri alternative:

[A1] **Nu există eveniment activ sau utilizatorul este restricționat** Sistemul afișează statistică prezențelor înregistrate și fluxul se termină.

[A2] **Utilizatorul refuză/amână confirmarea** Utilizatorul apasă „Nu acum” sau „Anulează”. Sistemul îl redirecționează către ecranul principal.

3.2.4 Administrează Prezență Manual

Descriere: Adminul poate modifica manual prezența unui utilizator la un eveniment (de exemplu, dacă a uitat să se marcheze).

Actor: Admin (principal)

Eveniment declanșator: Adminul selectează un eveniment pentru administrare.

Precondiții: Evenimentul există și adminul este autentificat.

Postcondiții: Prezența este actualizată și statisticile recalulate.

Flux principal:

Admin	Sistem
1. Selectează evenimentul	2. Afișează listă participanților și statusul lor
3. Modifică starea unui utilizator	4. Validează acțiunea
	5. Salvează modificarea
	6. Recalculează statisticile
	7. Afișează listă actualizată

3.2.5 Creare Categorii de Evenimente

Nume: Creare categorii de evenimente

Descriere: Adminul definește o nouă categorie (Curs, Seminar, Laborator etc.) cu setări de orar, fereastră de prezență și reguli de recurență.

Actor: Admin (principal)

Flux principal:

Admin	Sistem
1. Selectează „Creare categorie”	2. Afișează formularul
2. Completează datele	3. Verifică datele [A1]
	4. Salvează categoria
	5. Afișează mesaj de succes

3.2.6 Creare Evenimente

Descriere: Adminul creează o instanță concretă a unei categorii pentru o anumită dată, cu opțiuni de recurență.

Actor: Admin (principal) **Flux principal:**

Admin	Sistem
1. Selectează „Creare eveniment”	2. Afișează formularul
2. Completează datele	3. Validează datele [A1][A2]
	4. Salvează evenimentul
	5. Afișează mesaj de succes

Fluxuri alternative:

[A1] **Date invalide:** Sistemul marchează câmpurile greșite.

[A2] **Conflict de orar:** Sistemul avertizează despre suprapunere și permite modificarea.

De asemenea, aici trebuie să includem și diagrama UML a cazurilor de utilizare care reprezintă actorii, cazurile de utilizare și legăturile dintre toate elementele. Tipurile de relații sunt prezentate în tabelul de mai jos.

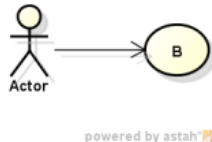
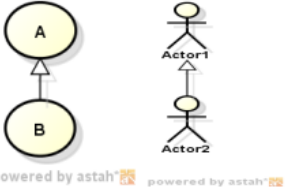
Denumire	Semantica	Sintaxa concreta UML
Relația de asociere	Describe comunicarea dintre actori și sistem	 powered by astah®
Relația de dependență (include)	Ori de câte ori va fi executat cazul A va fi executat și cazul B	 powered by astah®
Relația de dependență (extend)	Dacă se execută cazul A și este îndeplinită o condiție se execută și cazul B	 powered by astah®
Relația de generalizare	Are loc între actori sau între cazuri de utilizare	 powered by astah® powered by astah®

Figura 3.1: Tipuri de relații în diagrama UML de cazuri de utilizare

În figura 3.2 arătăm diagrama UML a cazurilor de utilizare ale sistemului PresSync.

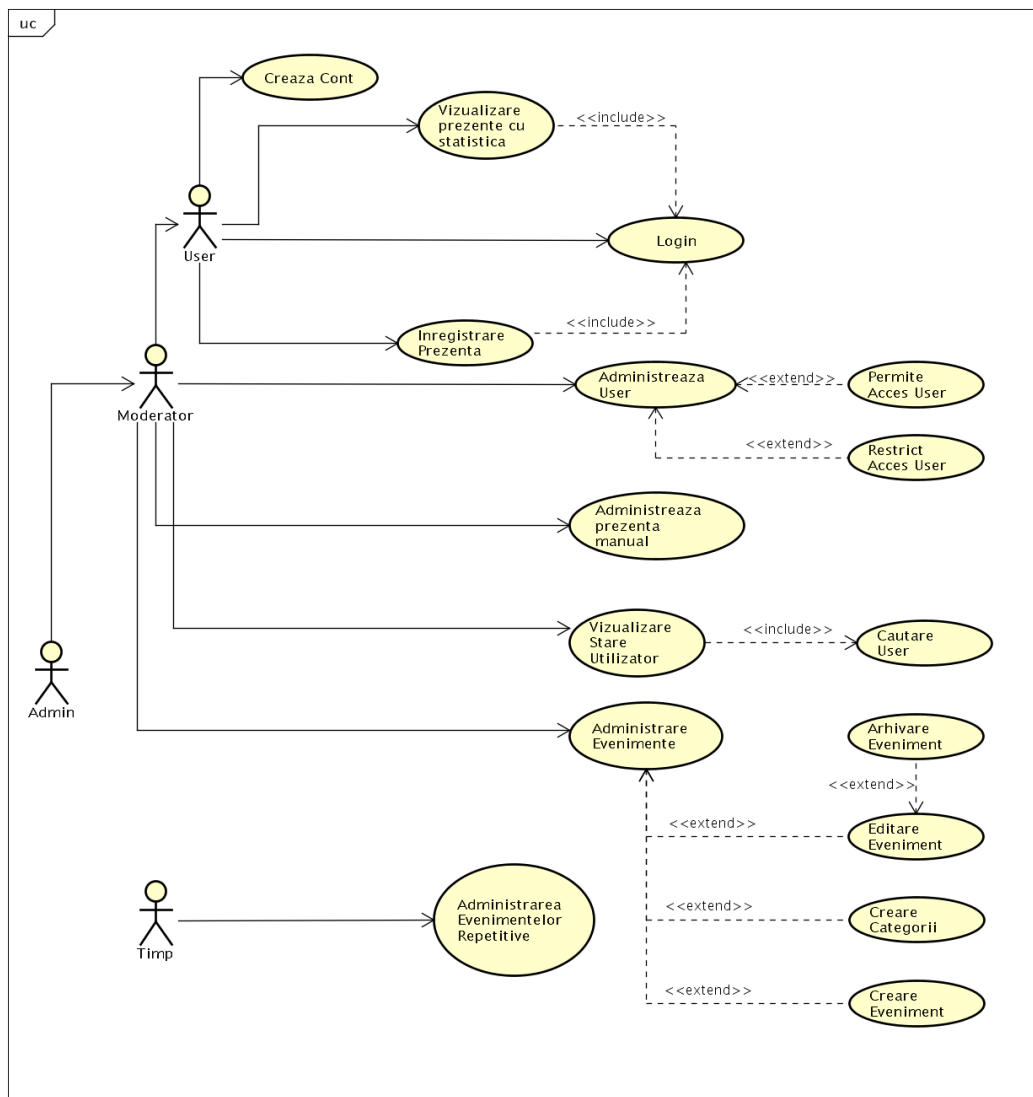
3.3 Diagrame de secvențe de sistem

3.3.1 Evenimente de sistem

Evenimentele de sistem apar datorită acțiunilor actorilor asupra sistemului software. Ele sunt de trei tipuri:

- ▷ De flux de date (create de acțiuni ce transmit date sau informații)
- ▷ De control (nu conțin date explicite)
- ▷ Temporare (evenimente generate de un cronjob)

Evenimentele de sistem identificate în urma analizei cazurilor de utilizare sunt prezentate în tabelul 3.1.



powered by Astah

Figura 3.2: Diagrama cazurilor de utilizare

Tip	Nr.	Descriere
Flux de date	1	Utilizatorul introduce datele de înregistrare sau logare și le trimite sistemului
Flux de date	2	Utilizatorul introduce codul primit pe adresa de mail electronică și îl trimite sistemului
Flux de date	3	Utilizatorul introduce numele unei categorii de eveniment în bara de căutare și trimite sistemului
Control	4	Sistemul marchează utilizatorul ca fiind prezent la evenimentul activ
Control	5	Sistemul autorizează accesul utilizatorului
Control	6	Sistemul verifică datele introduse de utilizator
Temporare	7	Sistemul creează evenimente în baza timpului setat de administrator
Temporare	8	Sistemul deschide prezența la evenimente dacă fereastra de prezență coincide cu timpul actual

Tabelul 3.1: Evenimente de sistem identificate în cazul sistemului PresSync

3.3.2 Diagrame de secvență de sistem

Aceste diagrame descriu interacțiunea dintre actori și sistemul software. Interacțiunea e descrisă de mesaje, care la rândul lor reprezintă comunicarea dintre două obiecte. În figura 3.3 prezint conceptele diagramelor de secvență de sistem și sintaxa în limbajul UML.

În figurile 3.3-3.7 sunt prezentate diagramele de secvență de sistem ale sistemului PresSync.

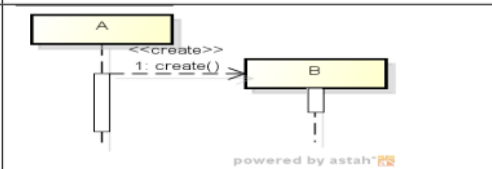
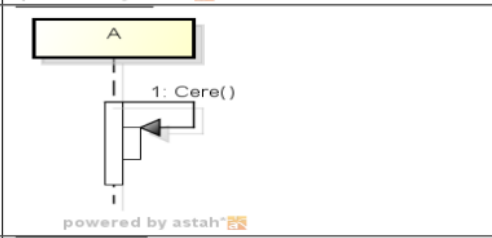
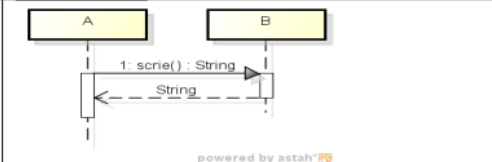
Linia de viață - indică existența unui obiect pe o perioadă de timp	
Mesajul <<create>> - în această perioadă este creat un obiect	
Mesajul <<destroy>> - descrie distrugerea unui obiect	
Activare - indică executarea unei operații de către obiectul însuși sau prin intermediul altor obiecte	
Mesajul <<return>> - indica tipul de date primit	

Figura 3.3: Prezentarea diagramelor de secvență de sistem

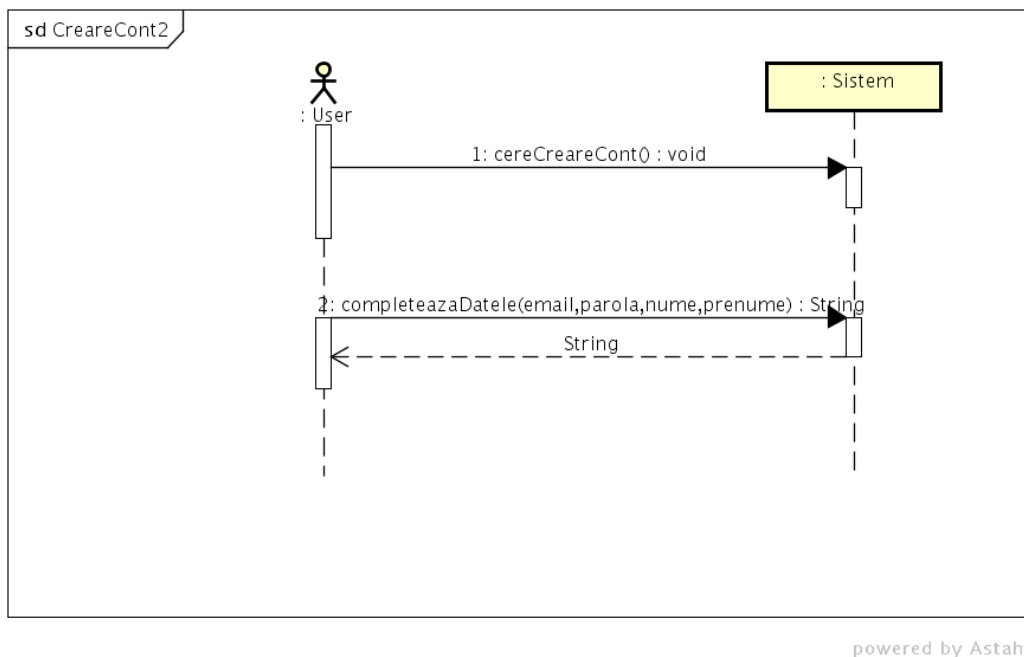
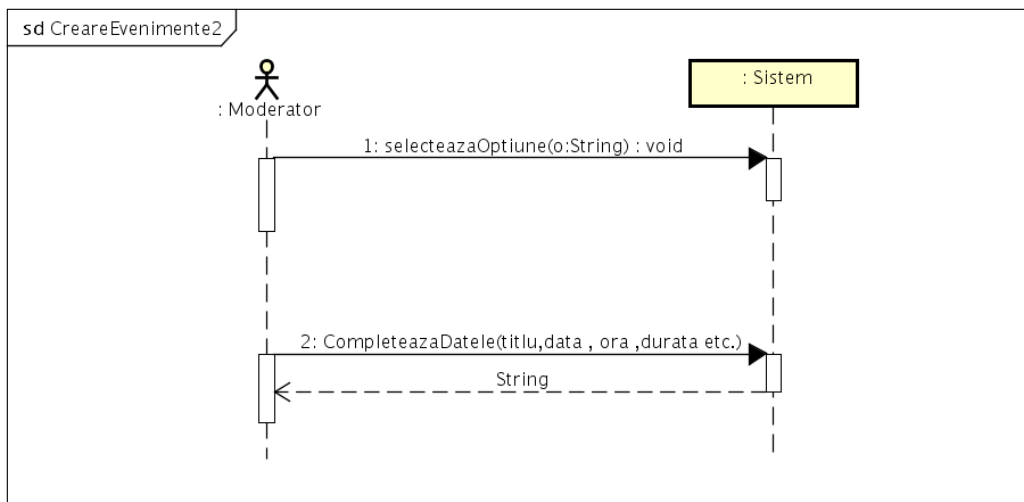
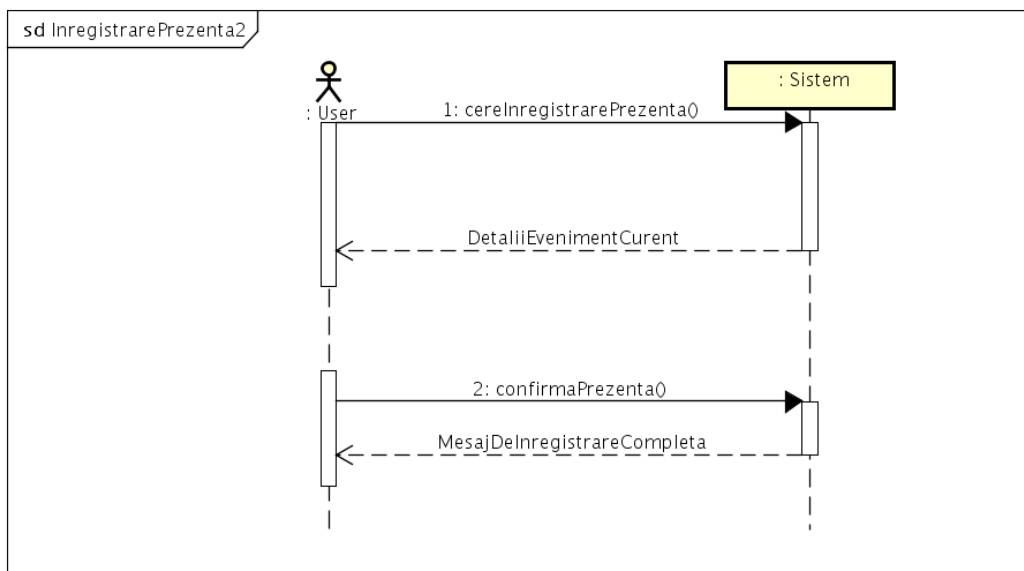


Figura 3.4: Diagrama de secvență — Creare cont



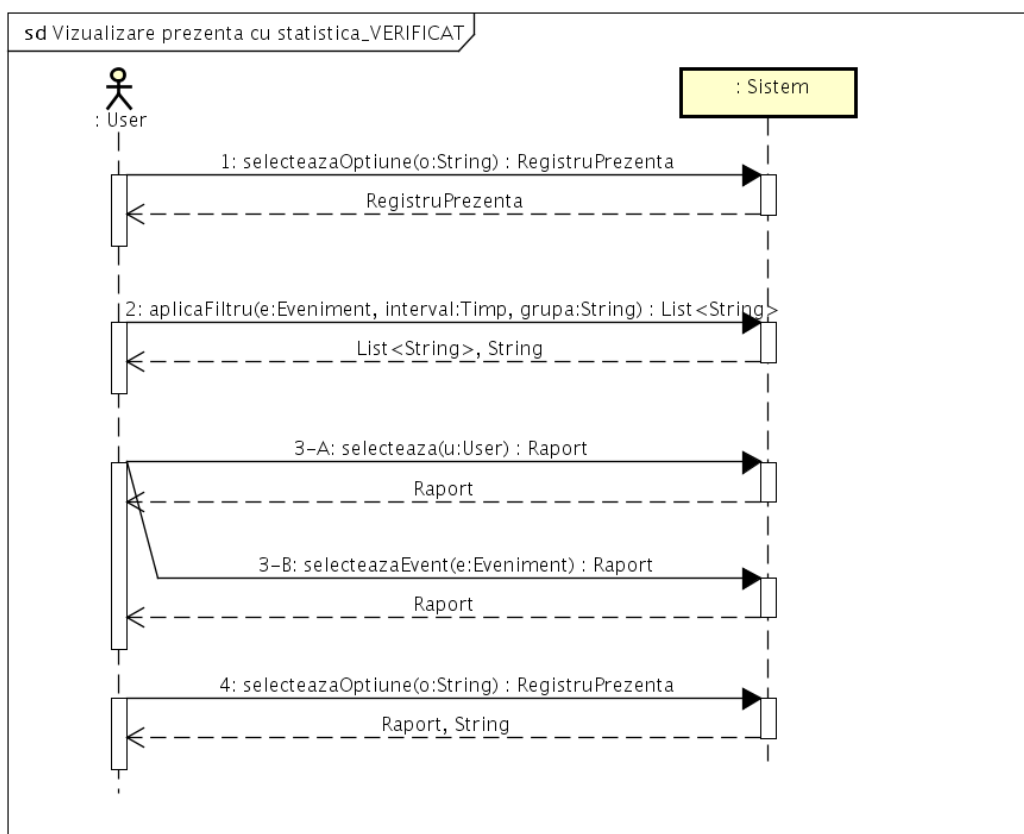
powered by Astah

Figura 3.5: Diagrama de secvență — Creare evenimente



powered by Astah

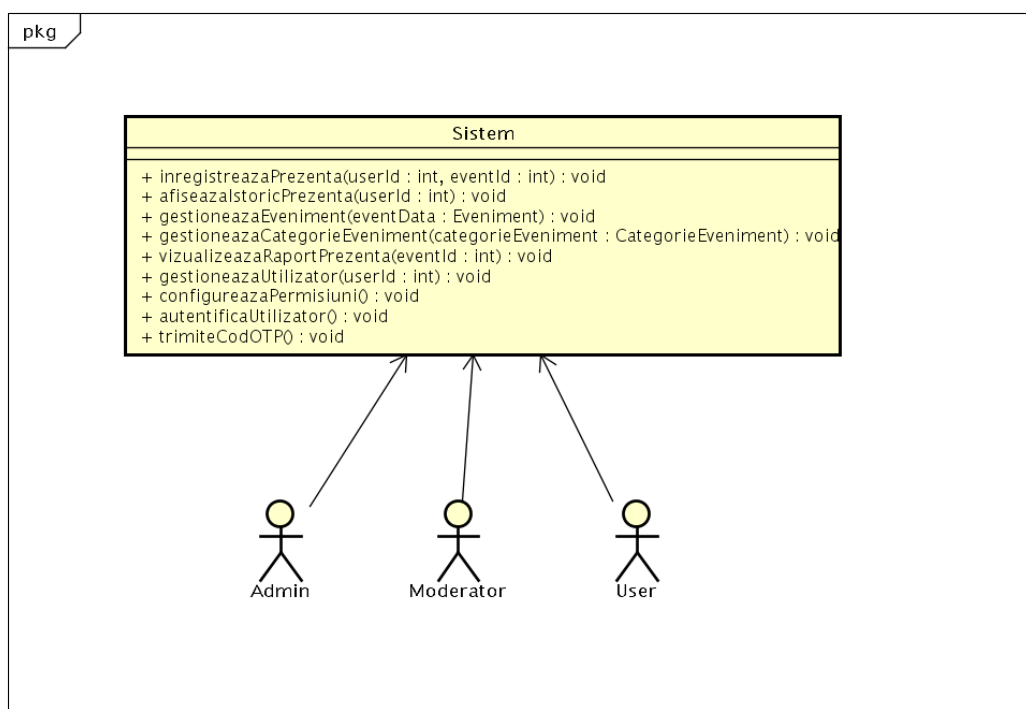
Figura 3.6: Diagrama de secvență — Înregistrare prezență



powered by Astah

Figura 3.7: Diagrama de secvență — Vizualizare prezență

Diagrama de context a sistemului este descrisă de acțiunile actorilor asupra sistemului și operațiile efectuate de acesta.



powered by Astah

Figura 3.8: Diagrama de context

3.4 Modelul domeniului

Crearea modelului de domeniu este un alt pas important care are la bază diagrama UML de clase. Diagrama UML de clase este creată pe baza următoarelor concepte: clasele modelează un grup de obiecte cu atribute similare, operații similare și au aceeași semantică; atributele sunt abstractizări ale proprietăților obiectelor claselor. Ele pot lua anumite valori. Operațiile descriu funcții sau schimbări ce pot folosi valorile atributelor intrinseci ale obiectelor clasei curente sau extrinseci ale obiectelor altor clase cu care clasa curentă comunică.

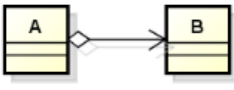
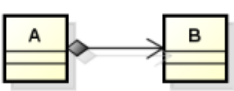
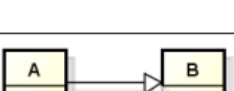
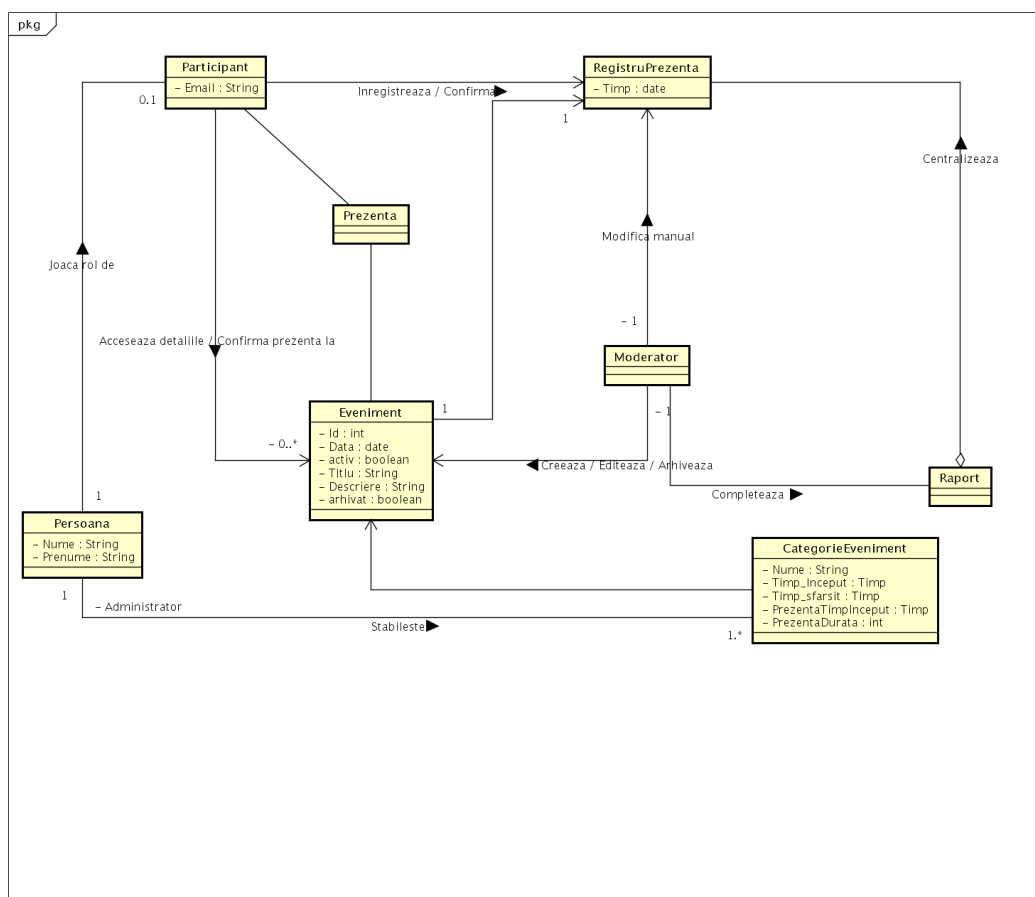
Relația de asociere	Reprezintă mulțimea de legături între obiectele claselor implicate	 powered by astah®
Relația de agregare	Reprezintă formarea unui lucru din părțile sale	 powered by astah®
Relația de compoziție	Este o relație de agregare mai puternică, deoarece un obiect nu poate exista fără celelalte obiecte agregat de care este legat.	 powered by astah®
Relația de generalizare /specializare	Face legătura între două clase în care una este supraclassa și cealalta este subclasa	 powered by astah®
Relația de dependență	Relație prin care dacă se modifică o clasa duce la afectarea altei clase cu care este legată	 powered by astah®

Figura 3.9: Explicarea relațiilor în modelul domeniului

Pentru a realiza diagrama UML trebuie să identificăm conceptele din descrierea problemei, ca mai apoi să le transformăm în clase sau atribute. Astfel, după analiza descrierii problemei am construit diagrama UML de clase prezentată mai jos.



powered by Astah

Figura 3.10: Diagrama de clase a modelului domeniului

Capitolul 4

Proiectarea și implementarea aplicației

Proiectarea software este o activitate de bază și este foarte importantă, ce urmează după identificarea și descrierea cerințelor software pe care le-am prezentat în capitolul anterior. Tot în această fază de dezvoltare a sistemului PresSync construim arhitectura software detaliată a sistemului.

4.1 Arhitecturi și stiluri arhitecturale

Aici reprezentăm modelul de organizare a sistemelor software. Modelul este compus din componentele sistemului și relațiile dintre componente împreună cu mediul în care se află și principiile care realizează proiectarea și evoluția arhitecturii.

Acest stadiu ne ajută să înțelegem construcția, managementul și evoluția unui sistem software.

Stilurile arhitecturale propun un vocabular de proiectare, împreună cu diferite constrângeri asupra modului de folosire a vocabularului și descrierea semanticii lui. Există o varietate de

stiluri, în secțiunea de mai jos voi prezenta câteva stiluri arhitecturale pe scurt.

- ▷ **Stilul arhitectural conducte și filtre** Filtrele sunt folosite pentru citirea fluxurilor de date, iar conductele pentru transmiterea datelor. Acest stil este reutilizabil și permite ca mai multe procese să fie îndeplinite simultan. Dezavantajul principal este organizarea dificilă a calculului.
- ▷ **Stilul arhitectural client-server** Acest stil separă procesele în două tipuri *proces client* și *proces server*. Aceste procese colaborează ca să realizeze o activitate. Avantajul este că clienții sunt independenți și oferă un control mai stabil asupra resurselor oferite de server.
- ▷ **Stilul arhitectural bazat pe evenimente.** Comunicarea componentelor evidențiază evenimentele. Deci o componentă transmite un semnal către altă componentă în legătură cu apariția unui eveniment la înregistrarea acestuia.
- ▷ **Stilul arhitectural pe straturi** Acesta este structurat pe nivele, fiecare nivel oferă servicii nivelului superior și acționează ca client pentru nivelul inferior. Avantajul este reutilizarea ușoară.
- ▷ **Stilul Model-View-Controller** Acesta se bazează pe izolarea componentelor logice față de cele ce țin de interfața utilizatorului. Astfel aplicația devine mult mai ușor de modificat pentru că nu afectează alte nivele.

4.2 Modele de proiectare folosite de atribuire a responsabilităților

Aceste modele sunt folosite pentru a descrie principiile fundamentale de proiectare orientată spre obiecte și atribuire a responsabilităților claselor. Responsabilitățile sunt obligații pe care o clasă le are. Mai jos urmează să enumăr câteva modele de proiectare folosite în construirea soluției mele și problemele pe care acestea le rezolvă.

Command-Query Separation (CQS) [2] este modelul arhitectural principal al aplicației. Acesta separă operațiile de modificare a stării sistemului (comenzi) de cele de citire (interogări). Interfața `Command<E, T>` definește metoda `execute(E entity)` pentru operații de tip CREATE, UPDATE, DELETE, iar interfața `Query<I, O>` definește metoda `execute(I input)` pentru operații de tip GET. Implementări concrete precum `CreateEventCategoryCommand`, `UpdateEventCommand` și `GetAllEventsQuery` sunt organizate în pachetele `CommandHandlers` și `QueryHandlers`. Acest model asigură separarea clară a responsabilităților și previne efectele secundare în operațiile de citire, fiind o concretizare a principiului *High Cohesion* din GRASP.

Strategy Pattern [3] este utilizat pentru gestionarea recurenței evenimentelor. Enum-ul `RepeatableType` definește strategiile suportate: `DAILY`, `WEEKLY`, `BIWEEKLY`, `MONTHLY` și `YEARLY`. În clasa `EventCategoryConfigService`, metoda `nextCandidateDate()` selectează strategia corespunzătoare printr-un `switch`, iar în `DailyLoaderScheduler`, metoda `shouldRunToday()` decide dacă o categorie de eveniment trebuie să ruleze într-o anumită zi. Acest model permite adăugarea de noi tipuri de recurență fără a modifica codul existent (*Open/Closed Principle*).

Observer Pattern [4] este implementat prin mecanismul de evenimente al Spring Framework. Clasa `EventCategoryChangedEvent` este un eveniment publicat de `CreateEventCategoryCommand` prin `ApplicationEventPublisher` atunci când o categorie de eveniment este modificată. Ascultătorul `DailyLoaderScheduler` reacționează prin adnotarea `@EventListener` și reîncarcă programul zilnic, asigurând desincronizarea componentelor (*Low Coupling*).

Chain of Responsibility [4] este folosit de Spring Security pentru procesarea cererilor HTTP. Filtrul `JwtAuthenticationFilter` face parte din lanțul de securitate și este poziționat înaintea filtrului `UsernamePasswordAuthenticationFilter`. Acesta extrage token-ul JWT din antetul cererii, îl validează și, dacă este valid, autentifică utilizatorul în contextul de securitate; în caz contrar, cererea este transmisă mai departe în lanț.

Adapter Pattern [4] este utilizat pentru integrarea entității `User` cu Spring Security. Clasa `User` implementează interfața `UserDetails` și definește metodele `getAuthorities()`, `isAccountNonExpired()` și `isEnabled()`, adaptând modelul intern al utilizatorului la sistemul de autentificare al framework-ului (*Protected Variations*).

Repository Pattern [4] este implementat prin Spring Data JPA. Interfețe precum `UserRepository`, `EventRepository` și `EventCategoryRepository` extind `JpaRepository` și oferă operații standard de acces la date, precum și metode personalizate prin adnotări `@Query`, abstractizând stratul de persistență (*Indirection*).

Facade Pattern [3] se regăsește în serviciile aplicației, precum `AuthenticationService` și `EventCategoryConfigService`. Acestea expun o interfață simplificată pentru operații complexe, coordonând mai multe componente interne și ascunzând detalii de implementare (*Pure Fabrication*).

Singleton Pattern [4] este asigurat implicit de containerul Spring prin adnotările `@Service` și `@Component`. Clasa `TodayScheduleCache` este un exemplu explicit — un cache volatil partajat la nivelul întregii aplicații, care stochează programul evenimentelor din ziua curentă.

DTO Pattern [2] este folosit pentru transferul datelor între straturile aplicației. Clase precum `CreateEventCategoryRequest`, `EventCategoryGetDTO`, `AttendanceGetDTO` și `UserCreatedDTO` asigură că entitățile interne nu sunt expuse direct către exterior, protejând integritatea datelor și permițând validări separate (*Protected Variations*).

4.3 Diagrame de interacțiune

Diagramele de interacțiune sunt folosite pentru a descrie modul în care interacționează componentele între ele într-un caz de utilizare. Ele se împart în două tipuri: de secvență și de comunicare. Diagramele de secvență arată ordinea transmiterii mesajelor, fiind mai flexibilă decât cealaltă. În continuare urmează să prezint diagramele pentru câteva cazuri de utilizare.

4.3.1 Creare Cont

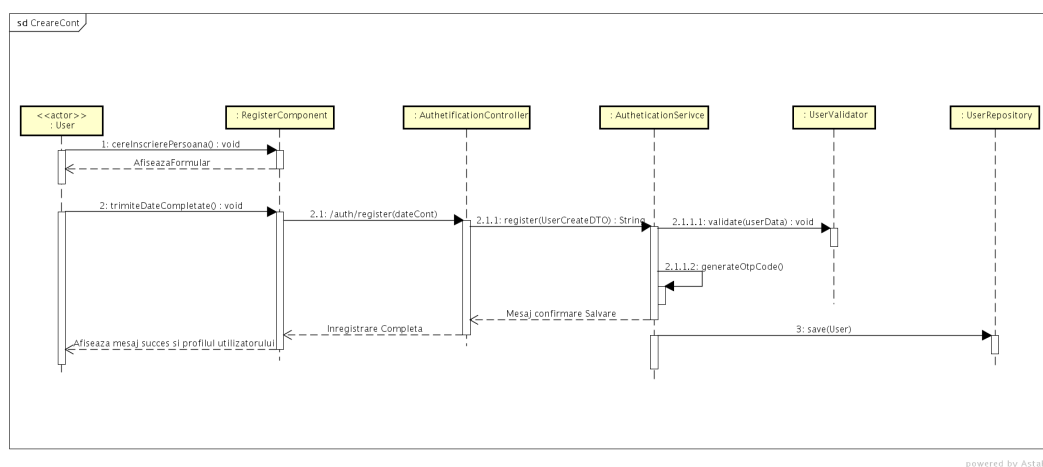


Figura 4.1: Diagrama de interacțiune — Creare cont

Conform diagramei de secvență, pentru implementarea acestui caz de utilizare am folosit clasa `AuthenticationController` pentru a separa fluxul de control în mai multe părți mai ușor de gestionat. Această clasă se folosește de `AuthenticationService` pentru a valida datele și pentru a crea obiectul de tip `User`. `AuthenticationController` conține metoda `register(userCreateDTO)` care apelează `AuthenticationService.registerUser`.

```
@PostMapping("/register")
public ResponseEntity<AuthenticationResponse> register(@RequestBody UserCreateDTO user) {
    return ResponseEntity.ok(service.register(user));
}
```

Figura 4.2: Implementare `AuthenticationController`

În `AuthenticationService` validăm datele mai întâi și după aceea creăm obiectul de tip `User` conform figurii 4.1.

În imaginile de mai jos putem vedea interfața grafică. În prima utilizatorul introduce un nume sau prenume greșit, în a doua utilizatorul introduce un email greșit, iar în ultima o parolă prea scurtă.

```

public AuthenticationResponse register(UserCreateDTO userCreateDTO) {
    userValidator.validate(userCreateDTO.getName(), userCreateDTO.getSurname(), userCreateDTO.getEmail());
    if (userCreateDTO.getPassword() == null || userCreateDTO.getPassword().length() < 6) {
        throw new IllegalArgumentException("Password must be at least 6 characters long");
    }

    User user = new User();
    user.setName(userCreateDTO.getName());
    user.setSurname(userCreateDTO.getSurname());
    user.setEmail(userCreateDTO.getEmail());
    user.setPassword(passwordEncoder.encode(userCreateDTO.getPassword()));
    user.setRole(UserRoles.USER);
    user.setActive(false);

    String otpCode = generateOtpCode();
    user.setMfaEnabled(true);
    user.setMfaCode(otpCode);
    user.setMfaExpiry(LocalDate.now().plusMinutes(5));
    repository.save(user);

    emailService.sendOtpCode(user.getEmail(), otpCode);

    String maskedEmail = maskEmail(user.getEmail());
    return AuthenticationResponse.mfaRequired(maskedEmail);
}

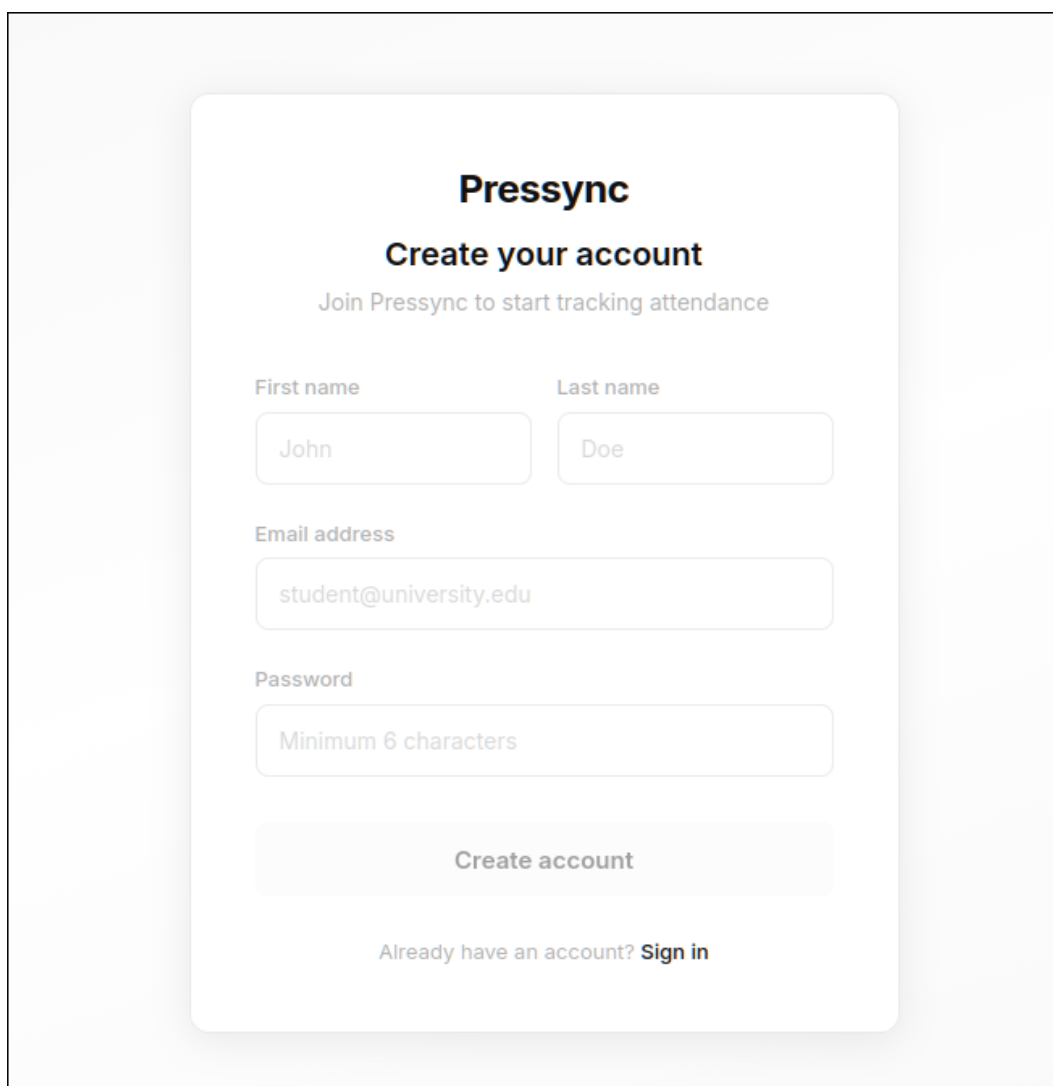
```

Figura 4.3: Implementare AuthenticationService

4.3.2 Creare Evenimente

Pentru acest caz de utilizare am implementat componenta frontend CreateCategoryComponent care, după completarea cu succes a formularului, apelează la endpointul de tip POST eventCategory/data.

Controllerul la rândul său apelează metoda execute a clasei CreateEventCategoryCommand.



The image shows a registration form for 'Pressync'. The form is centered on a light gray background. It has a white background with rounded corners and a subtle drop shadow. The form contains the following elements: a title 'Pressync' in bold, a subtitle 'Create your account', a descriptive text 'Join Pressync to start tracking attendance', two input fields for 'First name' (containing 'John') and 'Last name' (containing 'Doe'), an input field for 'Email address' (containing 'student@university.edu'), an input field for 'Password' (containing 'Minimum 6 characters'), a 'Create account' button, and a link 'Sign in' for users who already have an account.

Pressync

Create your account

Join Pressync to start tracking attendance

First name Last name

John Doe

Email address

student@university.edu

Password

Minimum 6 characters

Create account

Already have an account? [Sign in](#)

Figura 4.4: Interfața de înregistrare — formularul gol

The image shows a web form for creating a Pressync account. At the top, the title "Pressync" is displayed in a large, bold font, followed by the subtitle "Create your account" and the instruction "Join Pressync to start tracking attendance". Below this, a red-bordered box contains an error message: "Invalid name, surname, or email format", preceded by a red circle with a white 'x' icon. The form consists of several input fields: "First name" and "Last name" (both containing the text "123"), "Email address" (containing "thorn@example.com"), and "Password" (represented by a series of black dots). A "Create account" button is positioned below the password field. At the bottom, there is a link that says "Already have an account? Sign in".

Figura 4.5: Interfața de înregistrare — eroare nume invalid

The image shows a registration form with the following elements:

- First name:** A text input field containing "Vlad".
- Last name:** A text input field containing "Moraru".
- Email address:** A text input field containing "thorn". This field is highlighted with a red border, and a red error message "Please enter a valid email address." is displayed below it.
- Password:** A password input field with a yellow background and 15 black dots representing masked characters.
- Create account:** A light gray button with the text "Create account".
- Sign in:** A link with the text "Already have an account? Sign in".

Figura 4.6: Interfața de înregistrare — eroare email invalid

First name

Vlad

Last name

Moraru

Email address

thorn@example.com

Password

●●

Password must be at least 6 characters.

Create account

Already have an account? Sign in

Figura 4.7: Interfața de înregistrare — eroare parolă prea scurtă

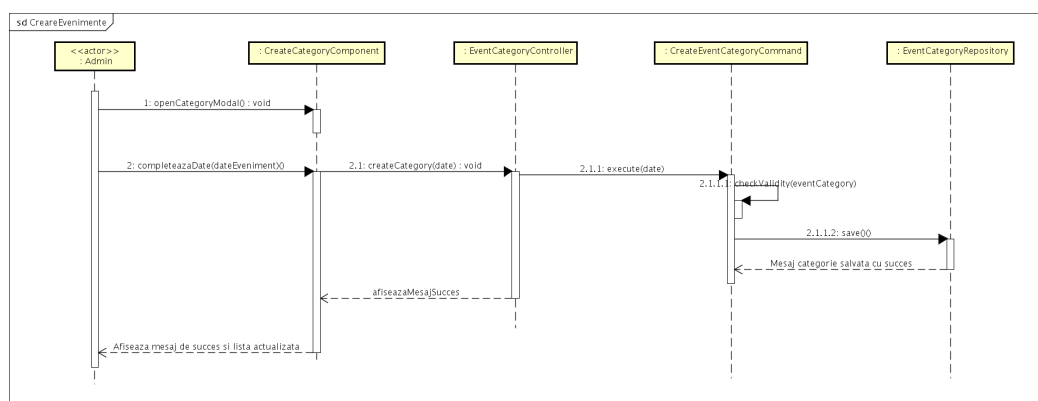


Figura 4.8: Diagrama de interacțiune — Creare evenimente

```

@PostMapping("/create") @ thorn
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<String> addEventCategory(@RequestBody CreateEventCategoryRequest eventCategoryRequest){
    return createEventCategoryCommand.execute(eventCategoryRequest);
}

```

Figura 4.9: Implementare EventCategoryController

```

public ResponseEntity<String> execute(CreateEventCategoryRequest request) {
    EventCategory entity = new EventCategory();
    entity.setName(request.name());
    entity.setStartingTime(request.startingTime());
    entity.setEndTime(request.endTime());
    entity.setAttendanceTimeStart(request.attendanceTimeStart());
    entity.setAttendanceDuration(request.attendanceDuration());
    entity.setRepeatable(request.repeatable());
    if (Boolean.TRUE.equals(request.repeatable())) {
        EventCategoryConfig config;
        if (request.configId() != null) {
            eventCategoryConfigService.enforceSingleCategoryPerConfig(request.configId(), categoryIdToIgnore: null);
            config = eventCategoryConfigRepository.findById(request.configId())
                .orElseThrow(() -> new IllegalArgumentException("Config not found"));
            eventCategoryConfigService.validateConfig(config);
        } else {
            config = new EventCategoryConfig();
            config.setRepeatableType(request.repeatableType());
            config.setRepeatsOnSpecificDay(request.repeatsOnSpecificDay());
            config.setBaseDate(request.baseDate() != null ? request.baseDate() : LocalDate.now());
            eventCategoryConfigService.validateConfig(config);
            config = eventCategoryConfigRepository.save(config);
        }
        entity.setCategoryConfig(config);
    } else {
        entity.setSpecificDate(request.baseDate() != null ? request.baseDate() : LocalDate.now());
    }

    checkValidity(entity);
    if (isCollidingWithSomething(entity)){
        throw new IllegalArgumentException("Colliding with something");
    };

    eventCategoryRepository.save(entity);
    applicationEventPublisher.publishEvent(new EventCategoryChangedEvent(entity));
    return ResponseEntity.ok( body: "{}");
}

```

Figura 4.10: Implementare CreateEventCategoryCommand

Această clasă creează obiectul de tip EventCategory și validează atât configul, adică partea ce ține de programarea evenimentelor următoare, să nu se ciocnească cu alte evenimente, cât și ca datele introduse să fie valide. În continuare prezint interfața grafică pentru acest caz de utilizare împreună cu toate erorile pe care aceasta le poate afișa.

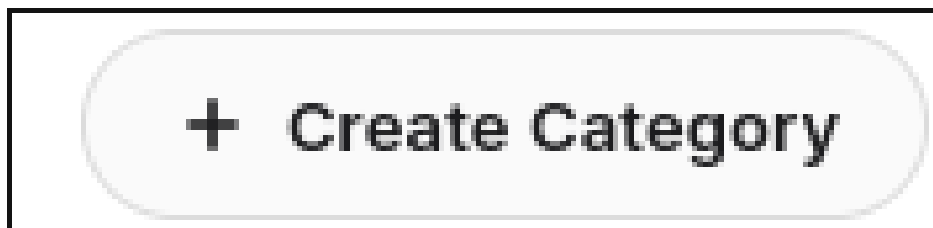


Figura 4.11: Interfața de creare categorii — butonul de creare

A screenshot of a 'Create New Category' dialog box. The dialog has a title bar with a close button (X). It is divided into two main sections: 'Event Schedule' and 'Attendance Window'. The 'Event Schedule' section includes a 'Category Name' field with the text 'Curs Blockchain', a 'Start Time' field with '12 : 00', and an 'End Time' field with '14 : 00'. Below these is a horizontal time range slider from 12:00 to 14:00. A 'Recurring event' toggle switch is located below the slider. The 'Attendance Window' section includes an 'Opens at' field with '12 : 30' and a 'Duration' field with a slider set to '20 min' (between 5 min and 90 min). The dialog has a light gray background and rounded corners.

Figura 4.12: Interfața de creare categorii — panoul de creare

Create New Category

×

Invalid attendance start time

Event Schedule

Category Name *

Curs Blockchain

Start Time *

12 : 00

End Time *

14 : 00

12:00

14:00

Opens 11:01 59 min before

☐ Recurring event

Attendance Window

Opens at *

11 : 01

Duration *

5 min

20 min

179 min

Figura 4.13: Interfața de creare categorii — eroare 1

Create New Category

×

Invalid attendance start time

Event Schedule

Category Name *

Curs Blockchain

Start Time *End Time *

12 : 0011 : 00

12:0011:00

Opens 11:00 60 min before

Recurring event

Attendance Window

Opens at *

11 : 00

Attendance must open before the event ends.

Figura 4.14: Interfața de creare categorii — eroare 2

Create New Category

×

Invalid start time or end time

Event Schedule

Category Name *

Curs Blockchain

Start Time *

12 : 00

End Time *

11 : 00

12:00

Opens 11:00 60 min before

11:00

Recurring event

Attendance Window

Opens at *

11 : 00

Attendance must open before the event ends.

Figura 4.15: Interfața de creare categorii — eroare 3

4.3.3 Înregistrare Prezență

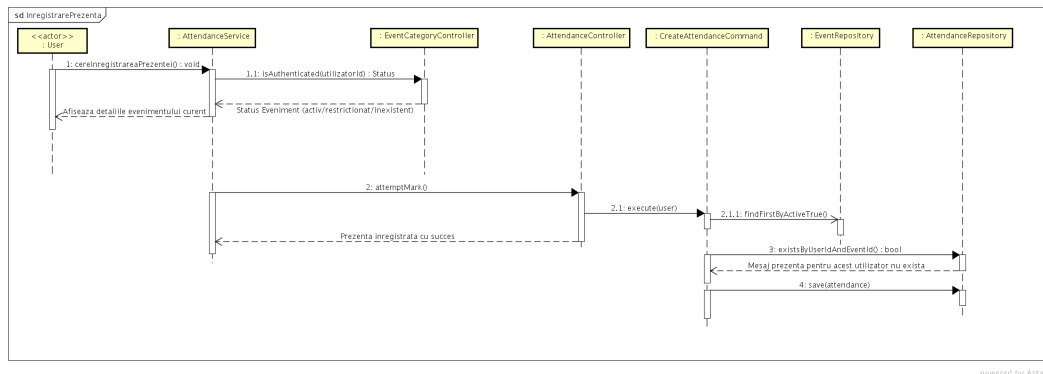


Figura 4.16: Diagrama de interacțiune — Înregistrare prezență

Pentru a implementa acest caz de utilizare, am creat un serviciu în frontend care verifică dacă există evenimente active cu fereastra de prezență ce coincide cu timpul actual. Dacă acestea coincid și utilizatorul este autentificat, serviciul face un apel la clasa `AttendanceController` din backend, care apelează la rândul ei `CreateAttendanceCommand`.

```
@PostMapping("/{mark}") @ thorn
@PreAuthorize("hasAnyRole('USER', 'MODERATOR', 'ADMIN')")
public ResponseEntity<String> createAttendance(@AuthenticationPrincipal User user) {
    return createAttendanceCommand.execute(user);
}
```

Figura 4.17: Implementare `AttendanceController`

`CreateAttendanceCommand` caută evenimentul activ, care poate fi unic, și verifică dacă există utilizatorul cu id-ul oferit de frontend. După ce toate verificările au fost cu succes, prezența este salvată folosind `AttendanceRepository` cu ajutorul metodei `save()`.

```

@Service  @thorn *
@RequiredArgsConstructor
public class CreateAttendanceCommand implements Command<User,String> {
    private final AttendanceRepository attendanceRepository;
    private final EventRepository eventRepository;

    @Override  @thorn *
    public ResponseEntity<String> execute(User user) {
        Attendance attendance = new Attendance();
        Event event = eventRepository.findFirstByActiveTrue().orElseThrow(()->
            new IllegalArgumentException("There is no active event for now"));
        LocalDateTime now = LocalDateTime.now().truncatedTo(ChronoUnit.MINUTES);

        if (attendanceRepository.existsByUserIdAndEventId(user.getId(), event.getId())) {
            throw new IllegalArgumentException("Attendance already exists");
        }

        LocalDateTime startWindow = event.getEventCategory().getAttendanceTimeStart().toLocalTime();
        int duration = event.getEventCategory().getAttendanceDuration().intValue();
        LocalDateTime endWindow = startWindow.plusMinutes(duration);

        if (now.isBefore(startWindow)) {
            throw new IllegalArgumentException("Too early! Attendance for " +
                event.getEventCategory().getName() + " starts at " + startWindow);
        }

        if (now.isAfter(endWindow)) {
            throw new IllegalArgumentException("Too late! The attendance window closed at " + endWindow);
        }

        attendance.setUser(user);
        attendance.setEvent(event);
        attendanceRepository.save(attendance);
        return ResponseEntity.ok().body("Successfully joined attendance");
    }
}

```

Figura 4.18: Implementare CreateAttendanceCommand

4.3.4 Vizualizare prezență cu statistici

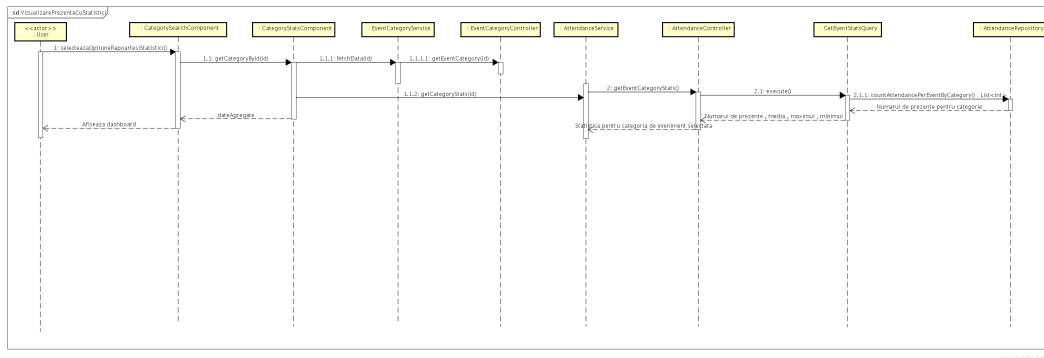


Figura 4.19: Diagrama de interacțiune — Vizualizare prezență cu statistici

Acest caz de utilizare a fost implementat în următorul mod. Utilizatorul caută evenimentul la care vrea să vadă prezența, folosind CategorySearchComponent. După selectarea evenimentului dorit este afișat un panel care prezintă statistica prezențelor la acel eveniment cu ajutorul lui CategoryStatsComponent.

Acest panel folosește EventCategoryService din frontend pentru a prelua datele din baza de date prin intermediul funcției getCategory(id) a clasei EventCategoryController.

Pentru a extrage statistica din backend, CategoryStatsComponent apelează metoda getCategoryStats din EventCategoryController care folosește AttendanceController pentru a calcula statistica pentru evenimentul cu id-ul dat, prin intermediul clasei GetEventStatsQuery. Acest query apelează metoda countAttendancePerEventByCategory a lui AttendanceRepository care returnează numărul de prezențe pentru categoria selectată.

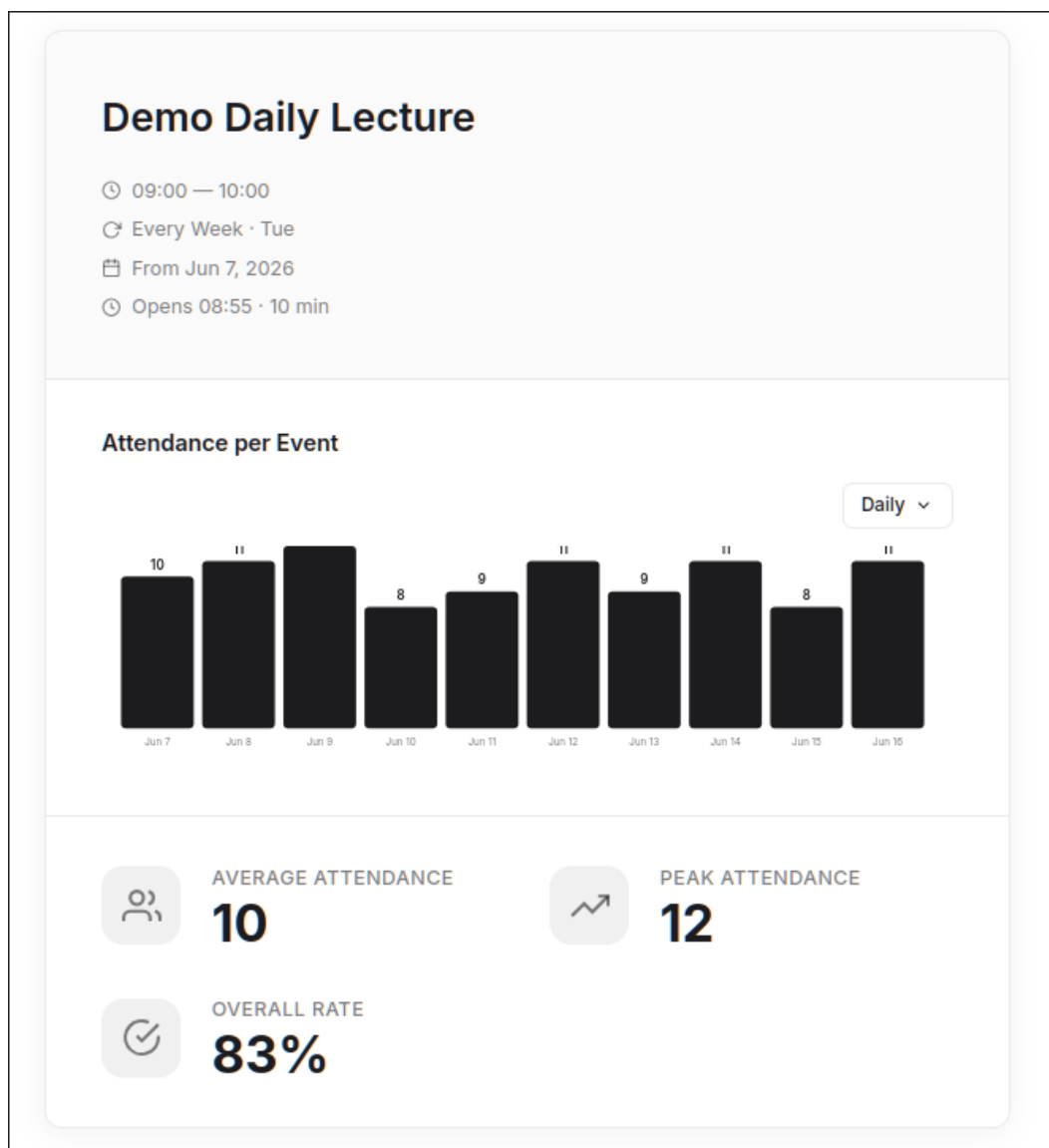


Figura 4.20: Vizualizare prezență — Admin

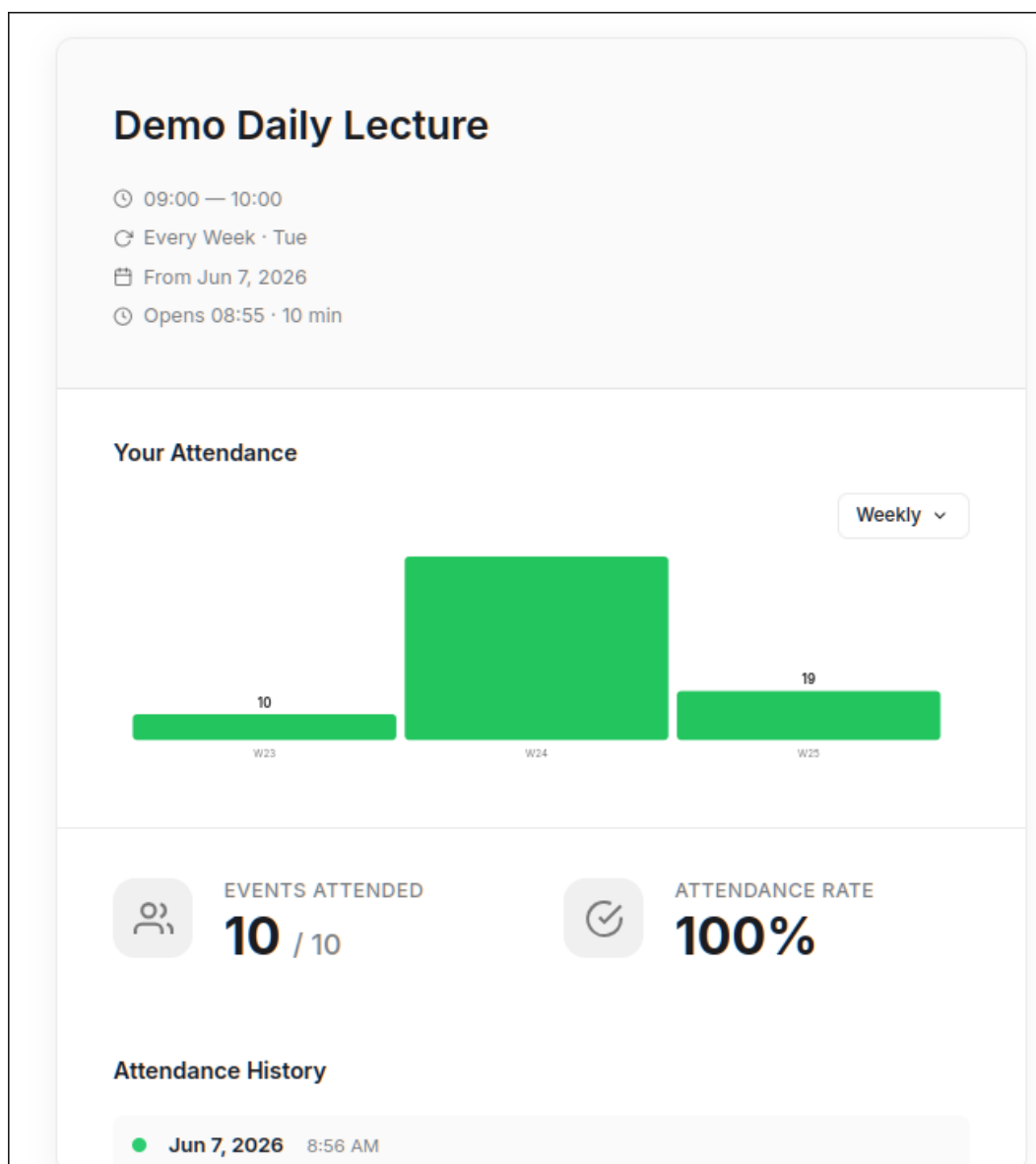


Figura 4.21: Vizualizare prezență — Săptămânal

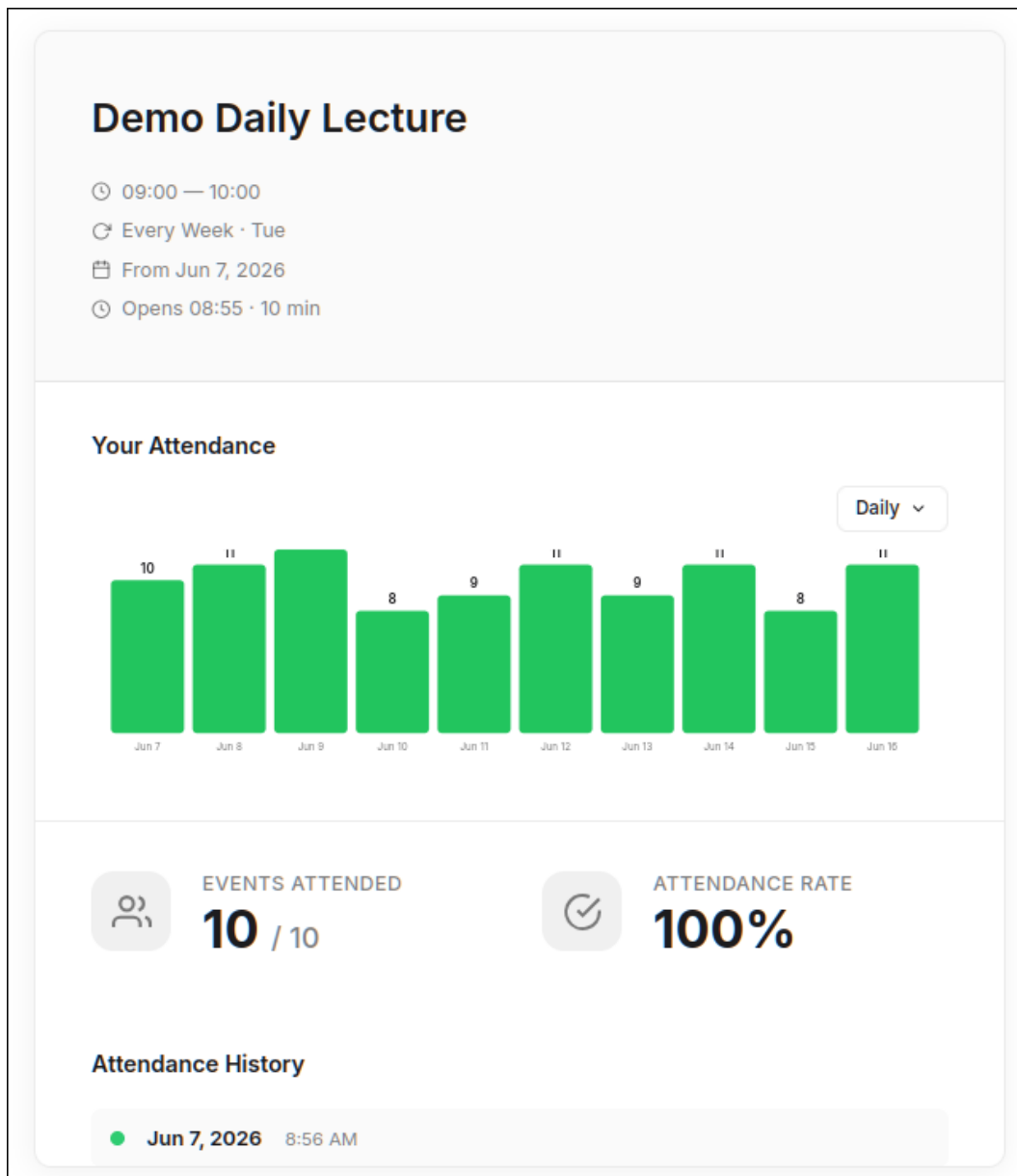


Figura 4.22: Vizualizare prezență — Zilnic

```
@GetMapping("/{stats/category/{categoryId}") @ thorn
@PreAuthorize("isAuthenticated()")
public ResponseEntity<EventCategoryStatsDTO> getEventCategoryStats(@PathVariable int categoryId) {
    return getEventCategoryStatsQuery.execute(categoryId);
}
```

Figura 4.23: Implementare EventCategoryController — getEventCategory

```

public ResponseEntity<EventCategoryStatsDTO> execute(Integer categoryId) {
    List<Long> counts = attendanceRepository.countAttendancePerEventByCategory(categoryId);
    List<EventAttendanceSummary> summaries = attendanceRepository.getEventAttendanceSummariesByCategory(categoryId);

    if (counts == null || counts.isEmpty()) {
        return ResponseEntity.ok(new EventCategoryStatsDTO( averageAttendance: 0L,
            maxAttendance: 0L, minAttendance: 0L, projectedNextAttendance: 0L, attendanceRate: 0L, summaries));
    }

    long max = Long.MIN_VALUE;
    long min = Long.MAX_VALUE;
    long sum = 0;

    double weightedSum = 0;
    double weightTotal = 0;

    for (int i = 0; i < counts.size(); i++) {
        long count = counts.get(i);

        if (count > max) max = count;
        if (count < min) min = count;
        sum += count;

        int weight = i + 1;
        weightedSum += count * weight;
        weightTotal += weight;
    }

    long average = Math.round((double) sum / counts.size());
    long projected = Math.round(weightedSum / weightTotal);
    long attendanceRate = max > 0 ? Math.round((double) average / max * 100) : 0;

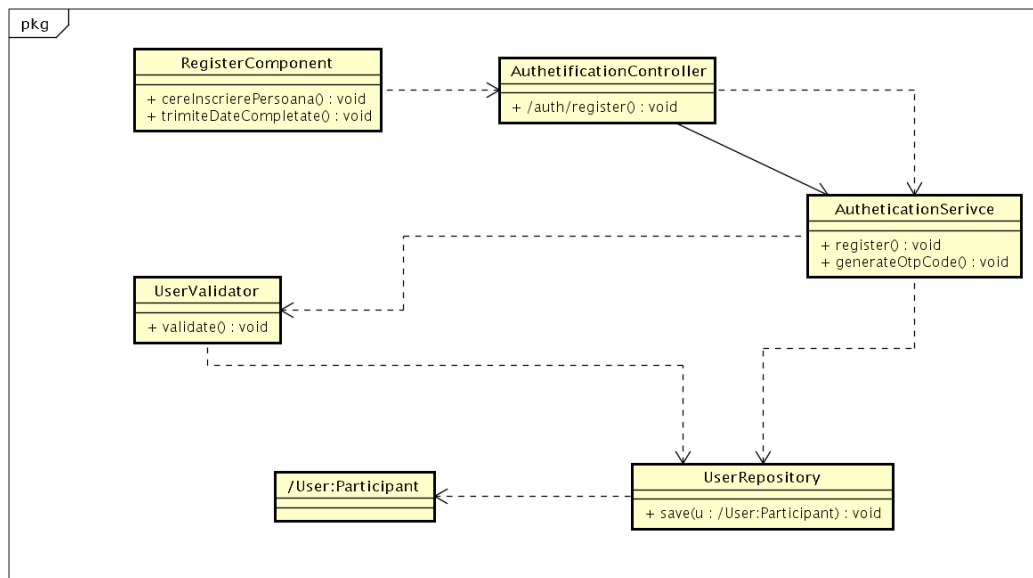
    EventCategoryStatsDTO stats = new EventCategoryStatsDTO(average, max, min, projected, attendanceRate, summaries);
    return ResponseEntity.ok(stats);
}

```

Figura 4.24: Implementare GetEventStatsQuery

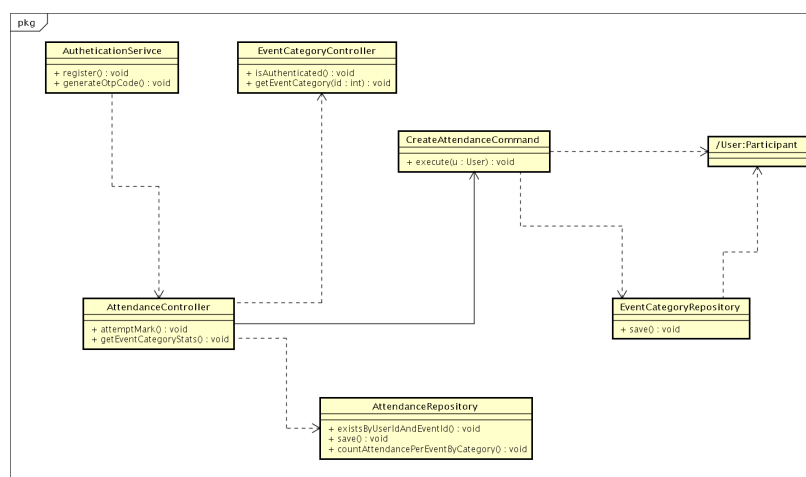
4.4 Diagrame de clase de proiectare

Diagramele de clase de proiectare ilustrează descrierea claselor și a interfețelor unui sistem cu tot cu relațiile dintre acestea. Ele conțin informațiile ce țin de clase cu atribute și operații, interfețe cu constante și operații și relațiile dintre acestea. Pentru a putea face aceste diagrame trebuie să identificăm clasele din diagramele de interacțiune și să adăugăm relațiile dintre clase. Mai jos voi prezenta diagramele pentru cazurile de utilizare: Creare cont, înscrisiere prezență.



powered by Astah

Figura 4.25: Diagrama de clase de proiectare — Creare cont



powered by Astah

Figura 4.26: Diagrama de clase de proiectare — Înscrisiere prezență

4.5 Modelarea bazei de date

Rezultatul după crearea entităților a fost structurata baza de date arătată în figura 4.27. Diagrama a fost generată automat cu ajutorul MySQL Workbench, pe baza entităților definite în codul sursă.

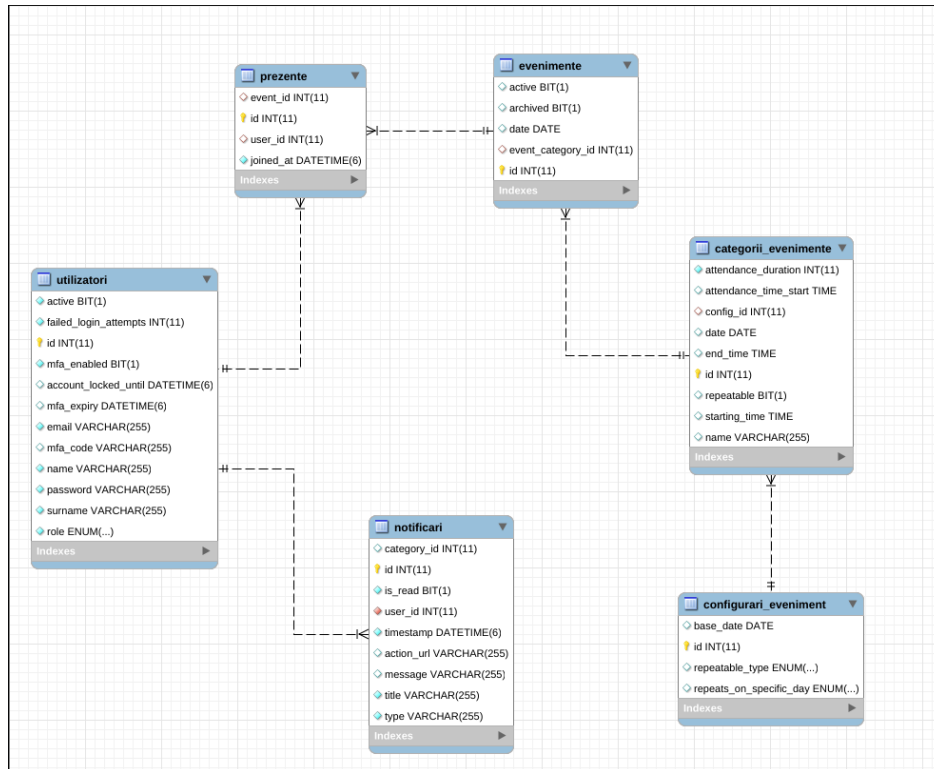


Figura 4.27: Structura bazei de date

Iar mai jos voi descrie puțin mai detaliat tabelele implementate.

Utilizatori — tabela principală pentru gestionarea conturilor. Conține câmpurile: `id` (cheie primară, auto-increment), `email` (unic, utilizat pentru autentificare), `password` (criptat BCrypt), `name`, `surname`, `role` (USER, MODERATOR, ADMIN), `active` (cont activ/dezactivat), `failed_login_attempts` și `account_locked_until` (suport pentru blocarea automată a contului), `mfa_enabled`, `mfa_code` și `mfa_expiry` (suport pentru autentificare multi-factor).

ConfigurăriEveniment — tabela care stochează configurația de recurență pentru evenimente. Câmpurile includ: `id` (cheie primară), `repeatableType` (DAILY, WEEKLY, BIWEEKLY, MONTHLY, YEARLY), `repeatsOnSpecificDay` (NO, MON-SUN) și `base_date` (data de referință pentru calculul recurenței). Este în relație One-to-Many cu tabela **CategoriiEvenimente**.

CategoriiEvenimente — definește tipurile de evenimente. Conține: `id`, `name` (numele categoriei), `startingTime` și `endTime` (orele de start și sfârșit), `attendanceTimeStart` și `attendanceDuration` (fereastra de marcare a prezenței, minim 5 minute), `repeatable` (flag boolean), `date` (data pentru evenimente nerecurente) și `config_id` (cheie străină către **ConfigurăriEveniment**).

Evenimente — reprezintă instanțe concrete ale unei categorii de evenimente. Câmpurile sunt: `id`, `event_category_id` (cheie străină către **CategoriiEvenimente**), `active`, `archived` și `date`. Este în relație Many-to-One cu **CategoriiEvenimente** și One-to-Many cu **Prezențe**.

Prezențe — înregistrează prezența utilizatorilor la evenimente. Conține: `id`, `user_id` și `event_id` (chei străine către **Utilizatori** și **Evenimente**, cu constrângere UNIQUE pentru a preveni dublarea), `joined_at` (timestamp-ul marcării prezenței, implicit CURRENT_TIMESTAMP).

Notificări — gestionează notificările către utilizatori. Câmpurile includ: `id`, `user_id` (cheie străină către **Utilizatori**), `type` (tipul notificării), `title`, `message`, `is_read` (flag citit/necitit), `timestamp`, `action_url` (URL opțional pentru acțiune) și `category_id` (link opțional către o categorie).

Capitolul 5

Concluzii

Pentru realizarea acestei aplicații, primul pas a fost descrierea și analiza problemei de gestionare a prezenței în contexte educaționale și organizaționale. În cadrul acestei analize am identificat limitările soluțiilor existente — sisteme RFID, coduri QR și tracking GPS — și am fundamentat arhitectural alegerea conectivității Wi-Fi la nivelul Stratului 2 al modelului OSI [1] ca mecanism de validare a prezenței fizice reale.

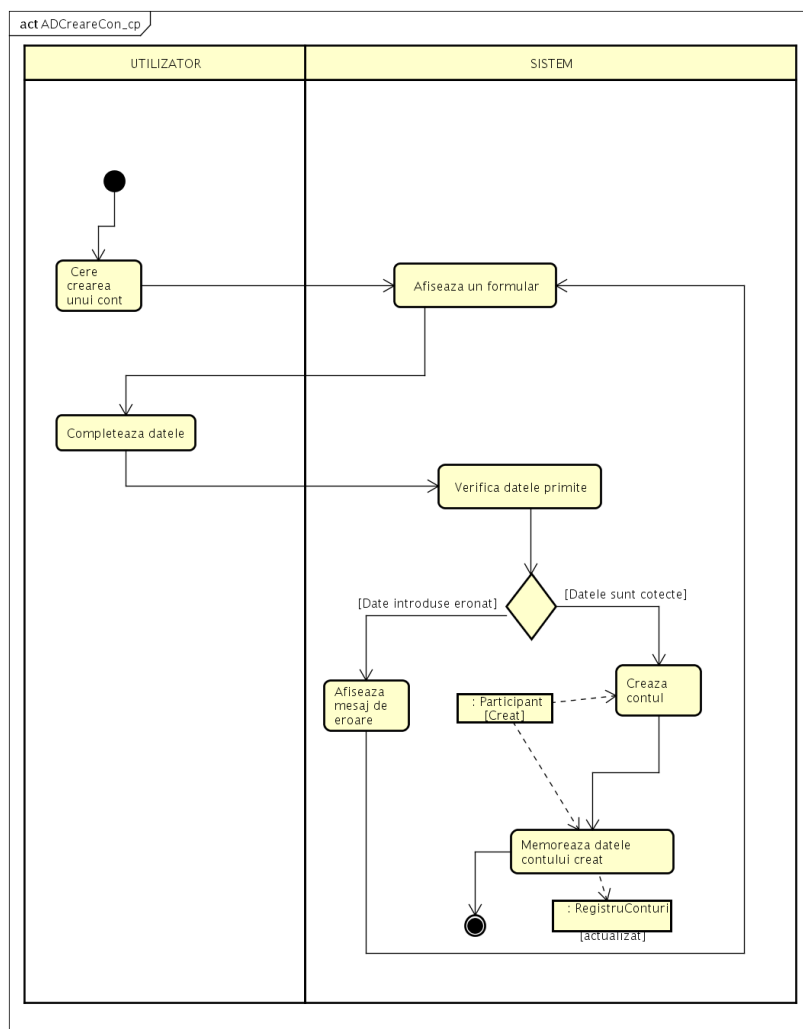
Pe baza acestei descrieri am urmat etapele esențiale în crearea aplicației software. În etapa de analiză am realizat documentul de cerințe, care conține actorii sistemului, cerințele funcționale F1–F5 și cerințele nefuncționale CN1–CN5. Tot în cadrul acestei etape am construit modelul domeniului, diagrama cazurilor de utilizare și diagramele de secvență de sistem, utilizând programul Astah pentru reprezentarea acestora.

Etapa de proiectare și implementare a presupus construirea diagramelor de interacțiune pentru cazurile de utilizare principale: Creare Cont, Creare Evenimente, Înregistrare Prezență și Vizualizare Prezență cu Statistici. Arhitectura aplicației este organizată conform principiului Command-Query Separation, completat de patternuri precum Strategy, Observer și Repository, implementate în Java cu Spring Framework. Interfața utilizator a fost dezvoltată ca aplicație web, accesibilă prin portalul captiv al routerului OpenWRT.

Sistemul PresSync a fost testat pe toate cazurile de utilizare definite și a furnizat rezultate corecte, demonstrând că validarea prezenței fizice prin conectivitate Wi-Fi locală reprezintă o alternativă viabilă, sigură și accesibilă față de soluțiile existente pe piață.

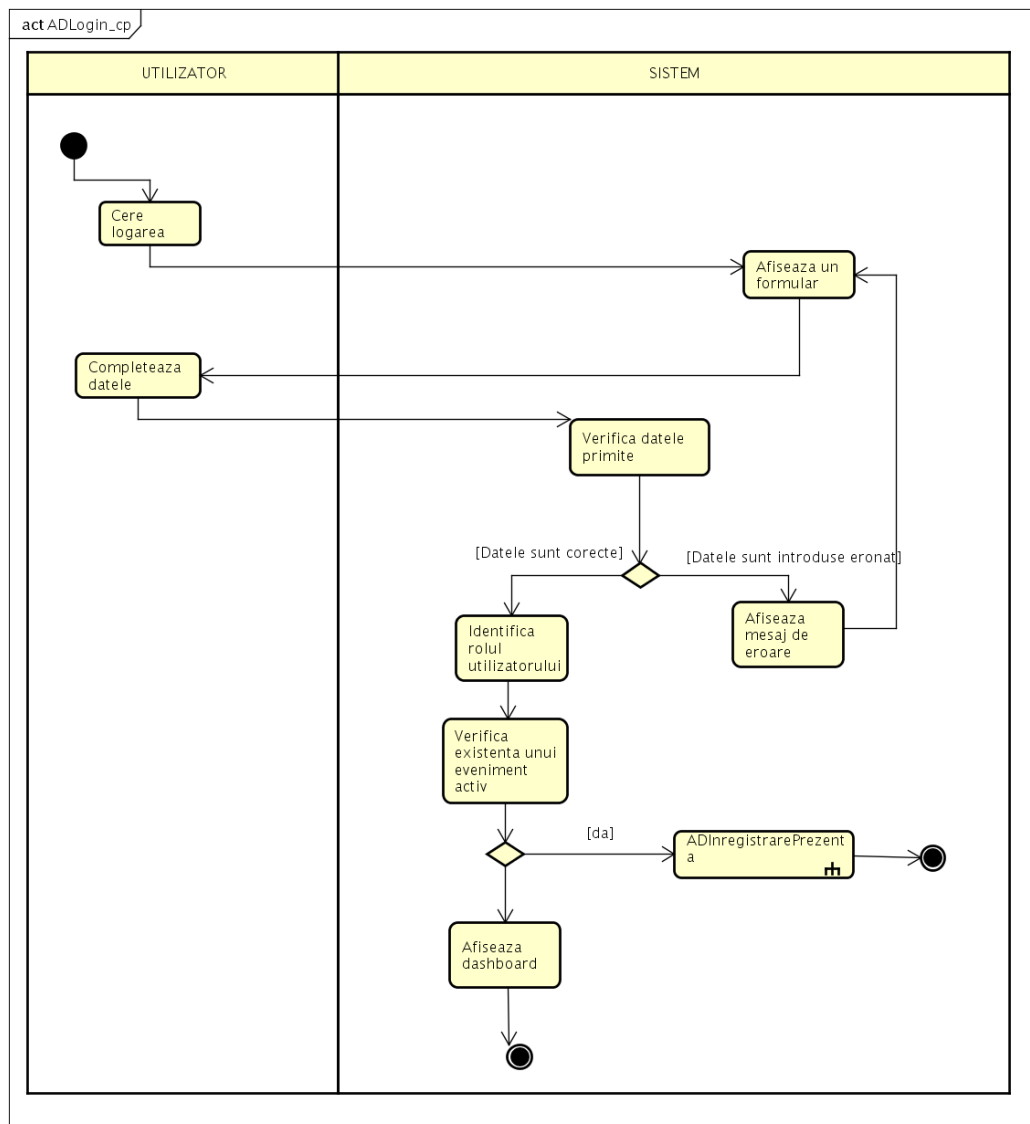
Capitolul 6

Anexe



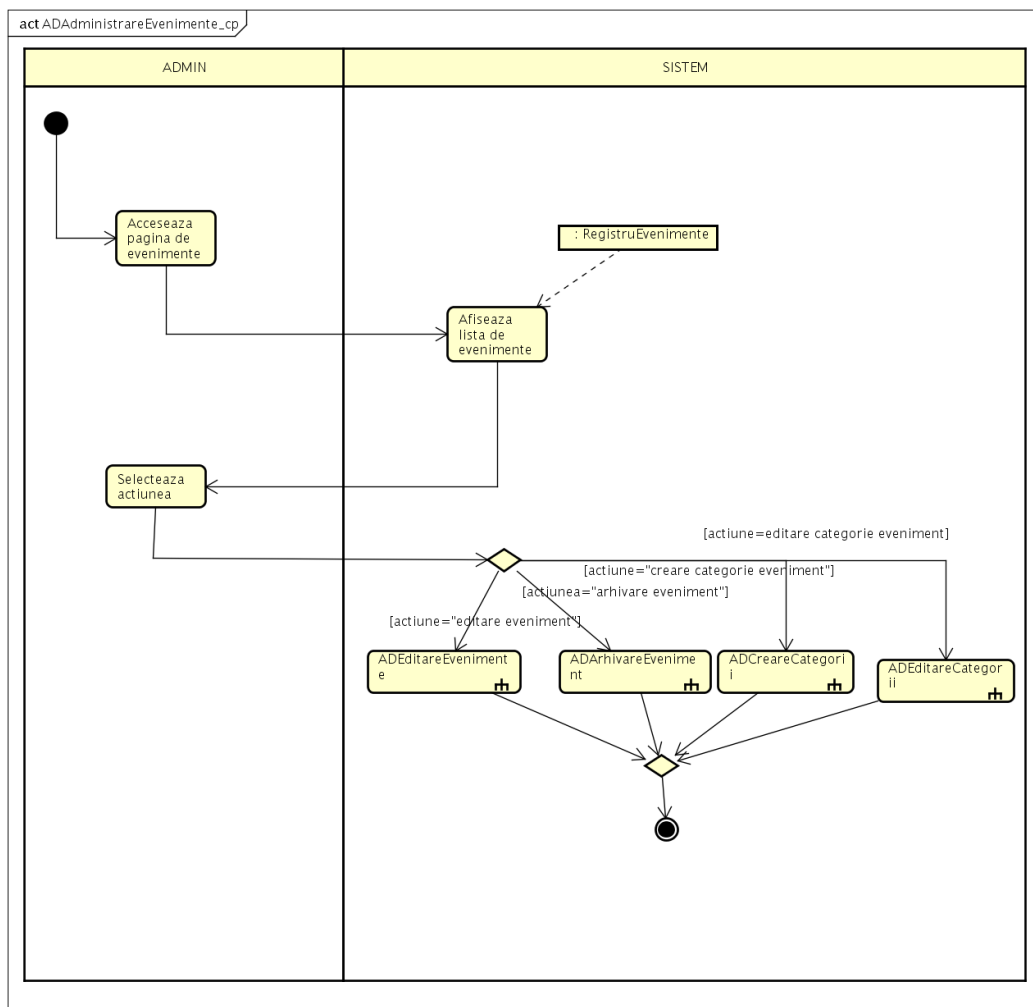
powered by Astah

Figura 6.1: Diagrama de activitate — Creare cont



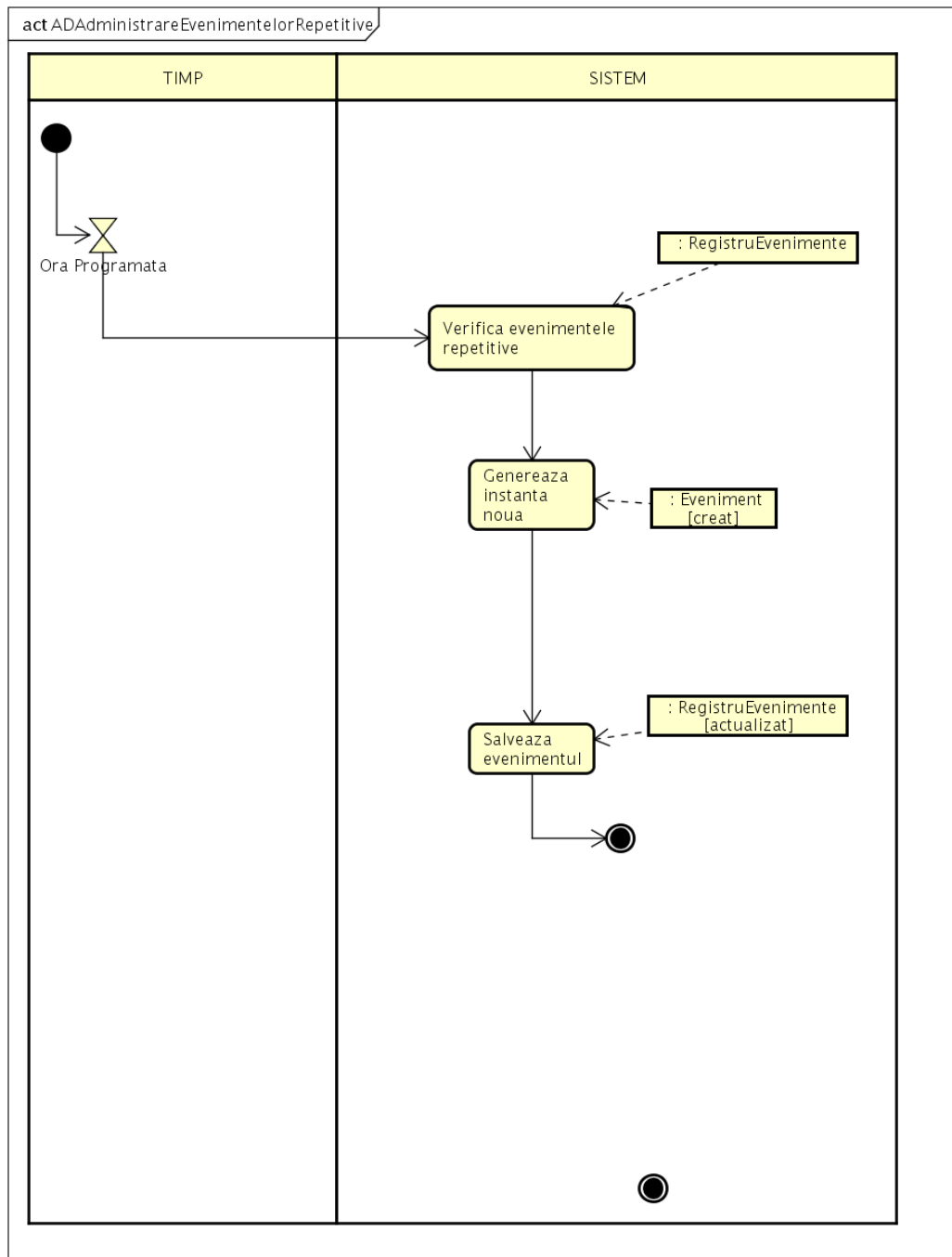
powered by Astah

Figura 6.2: Diagrama de activitate — Autentificare



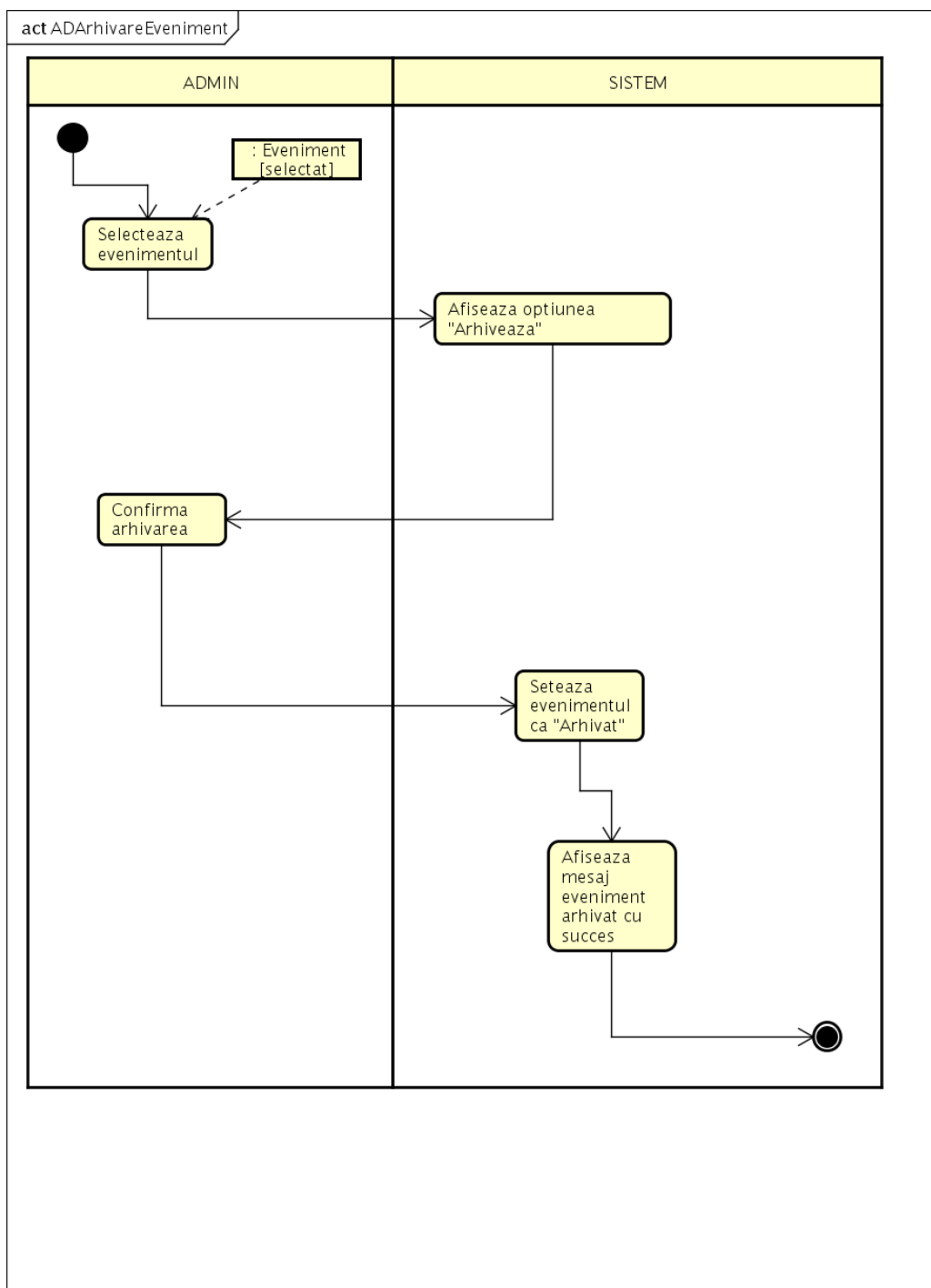
powered by Astah

Figura 6.3: Diagrama de activitate — Administrare evenimente



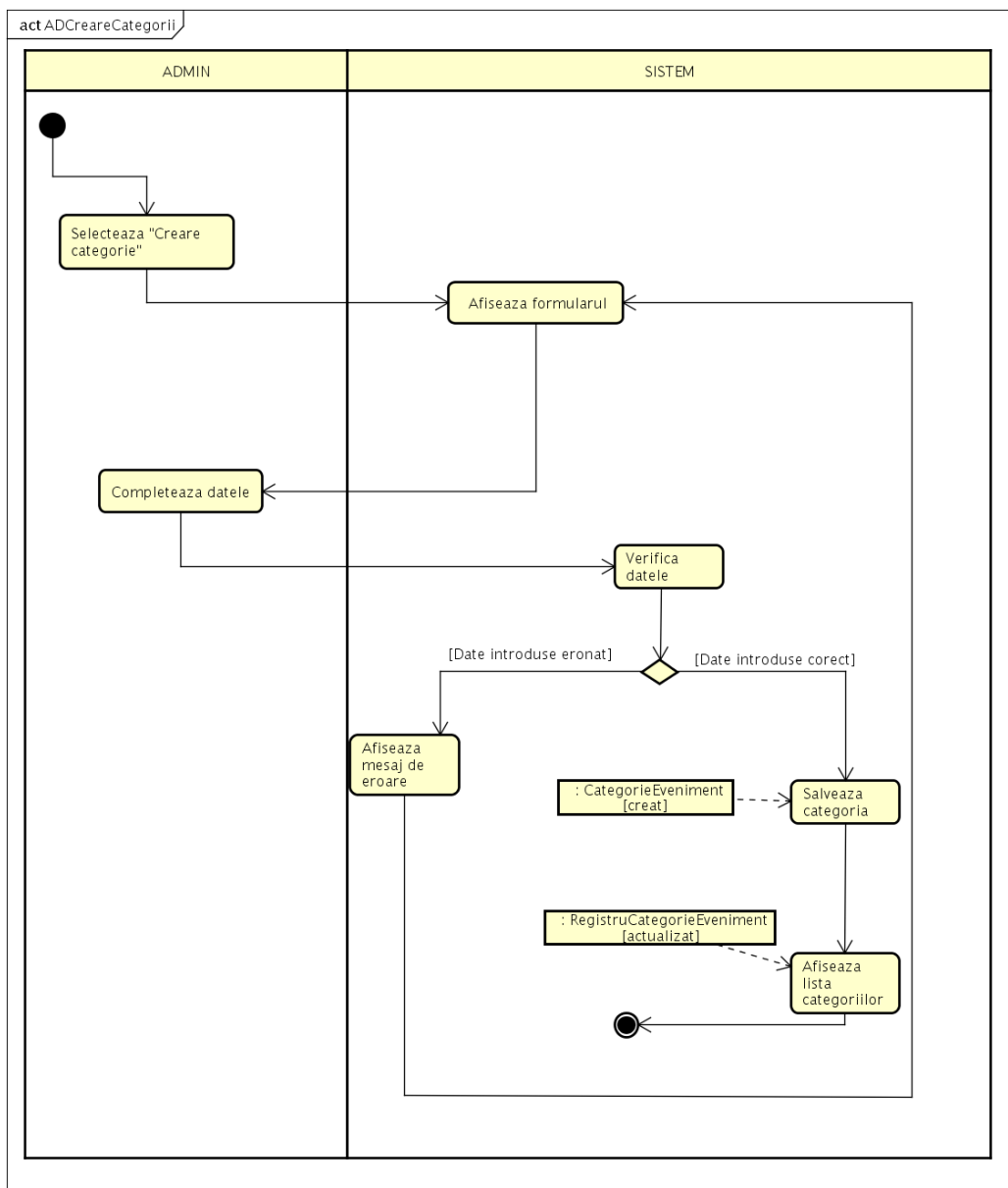
powered by Astah

Figura 6.4: Diagrama de activitate — Administrare evenimente repetitive



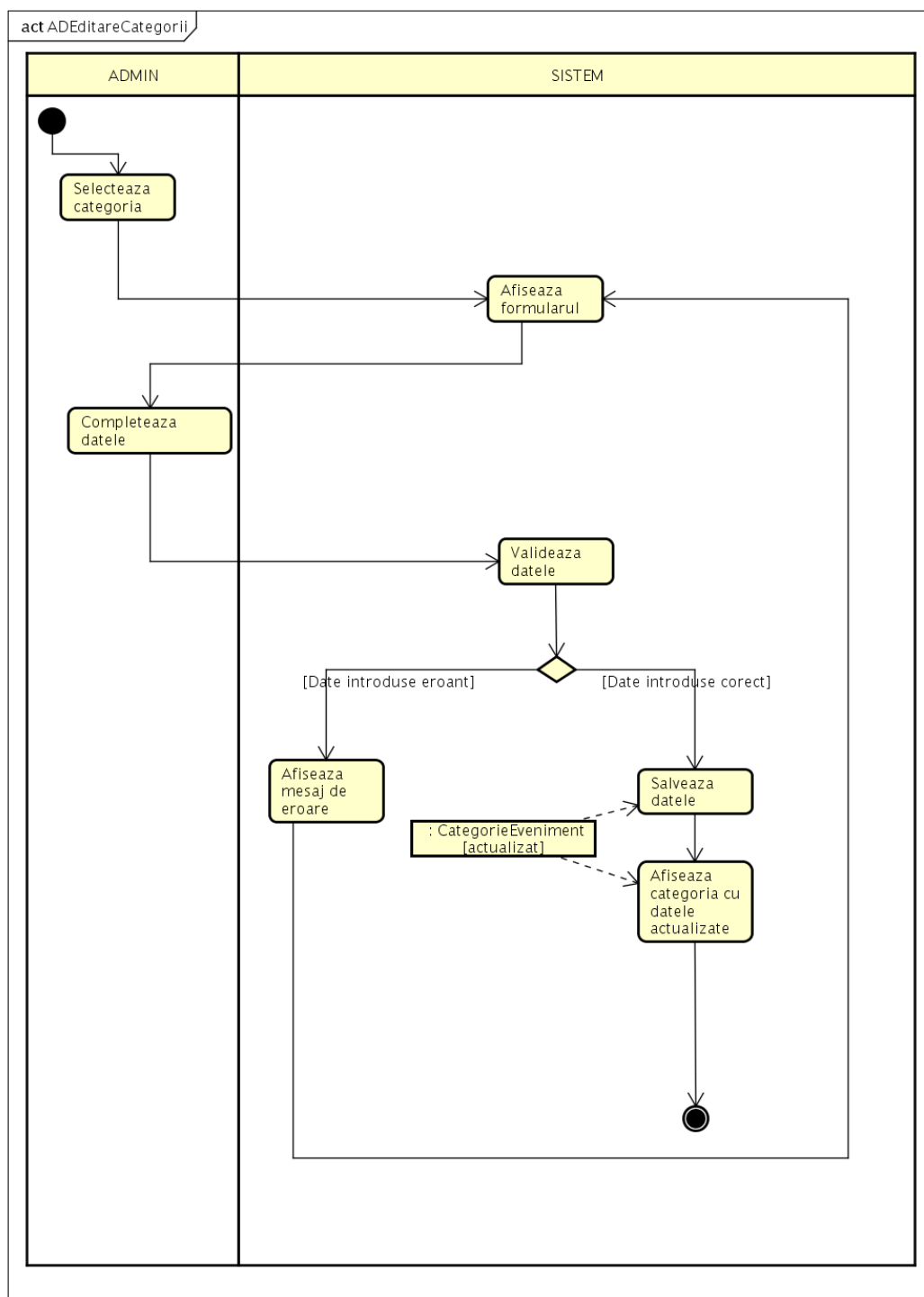
powered by Astah

Figura 6.5: Diagrama de activitate — Arhivare eveniment



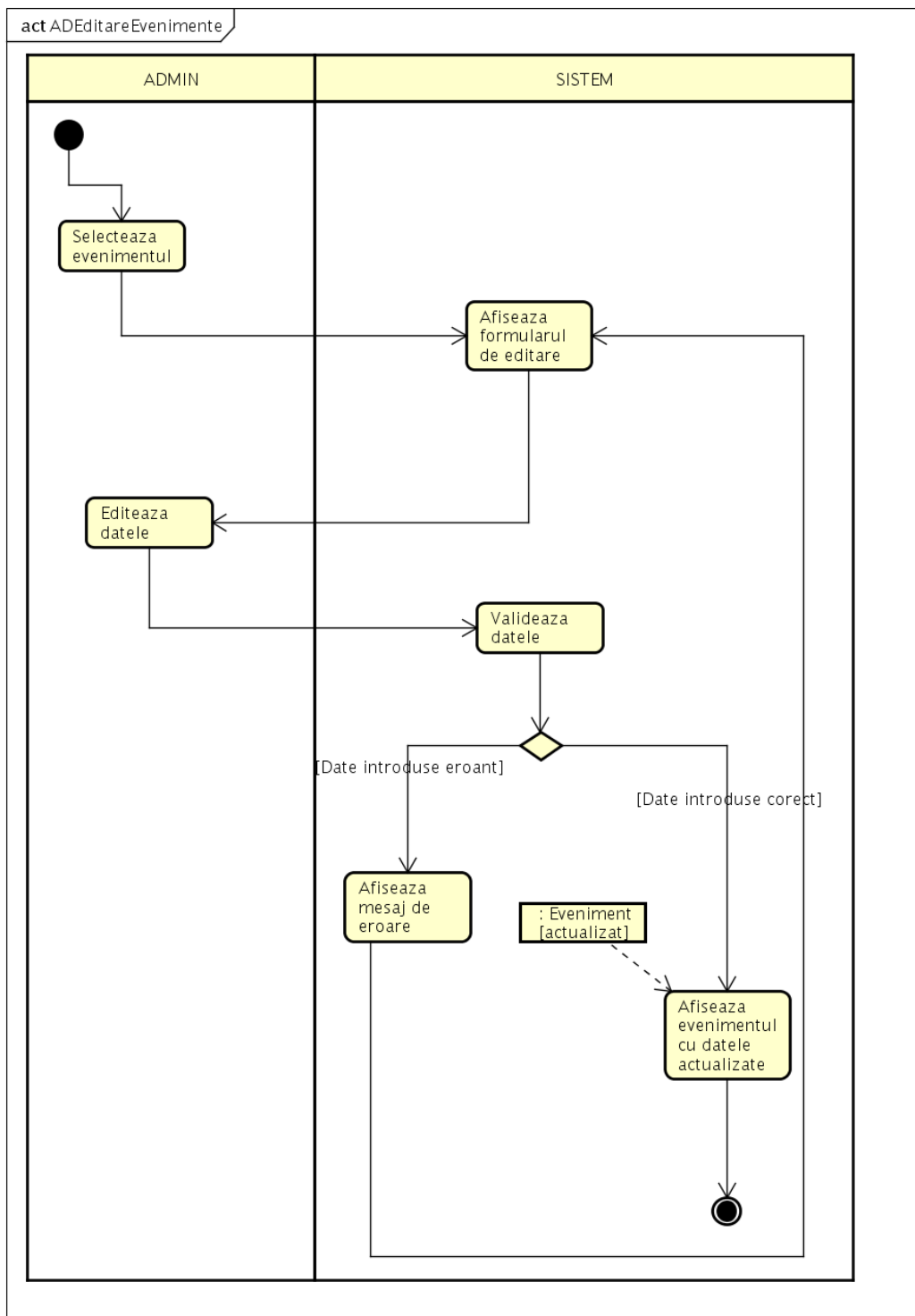
powered by Astah

Figura 6.6: Diagrama de activitate — Creare categorii



powered by Astah

Figura 6.7: Diagrama de activitate — Editare categorii



powered by Astah

Figura 6.8: Diagrama de activitate — Editare evenimente

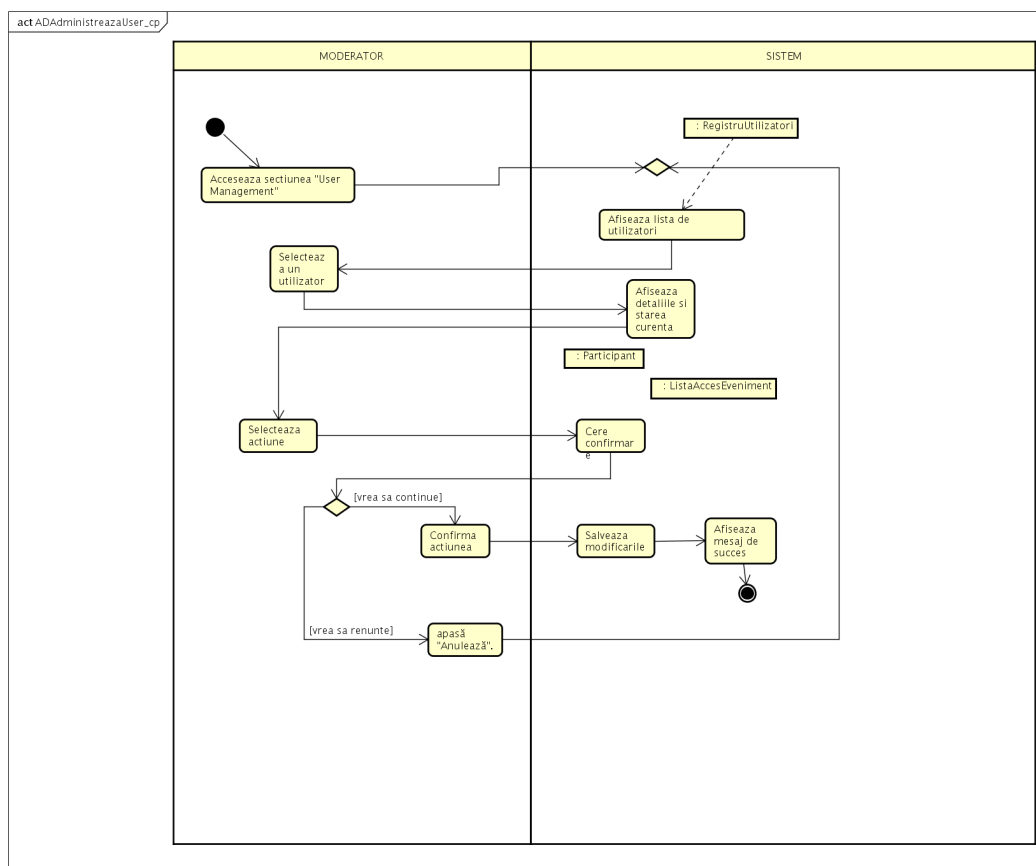


Figura 6.9: Diagrama de activitate — Administrare utilizator

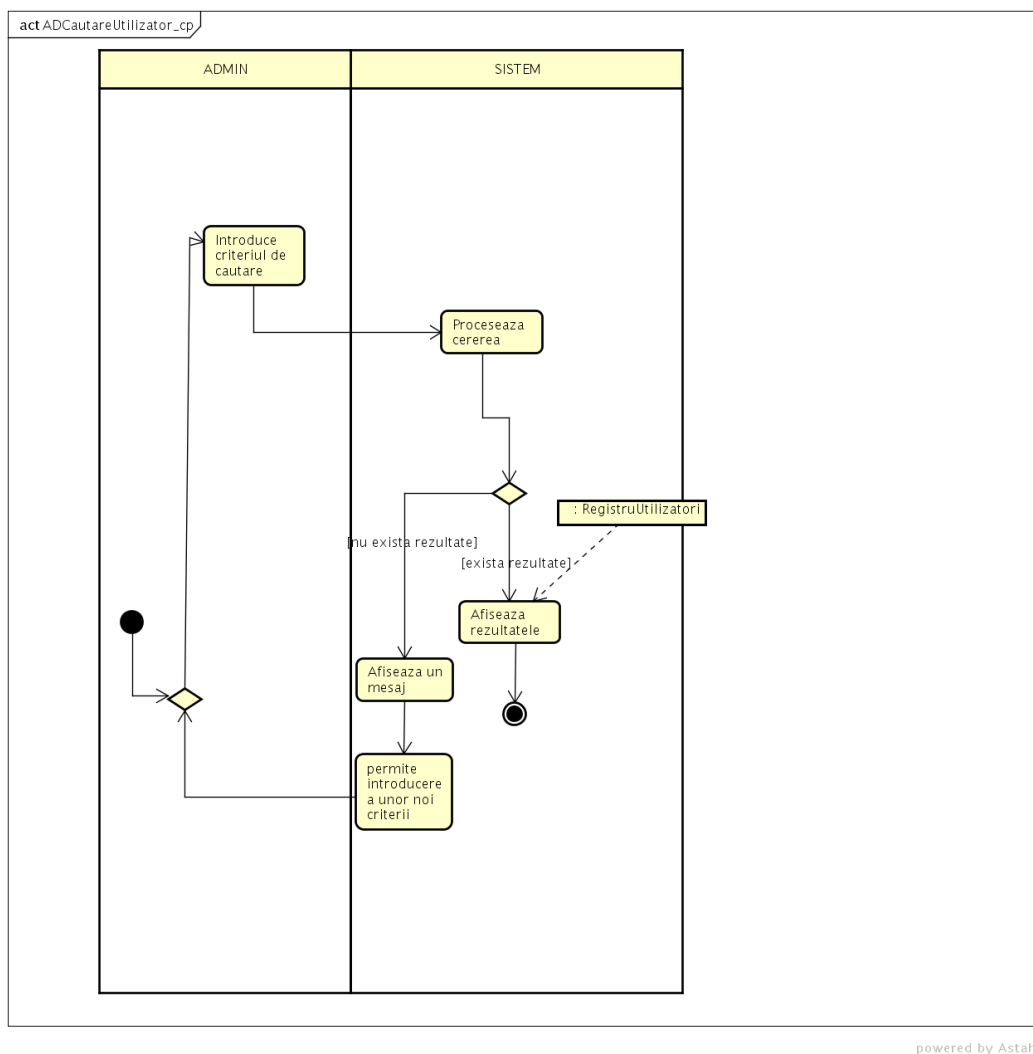
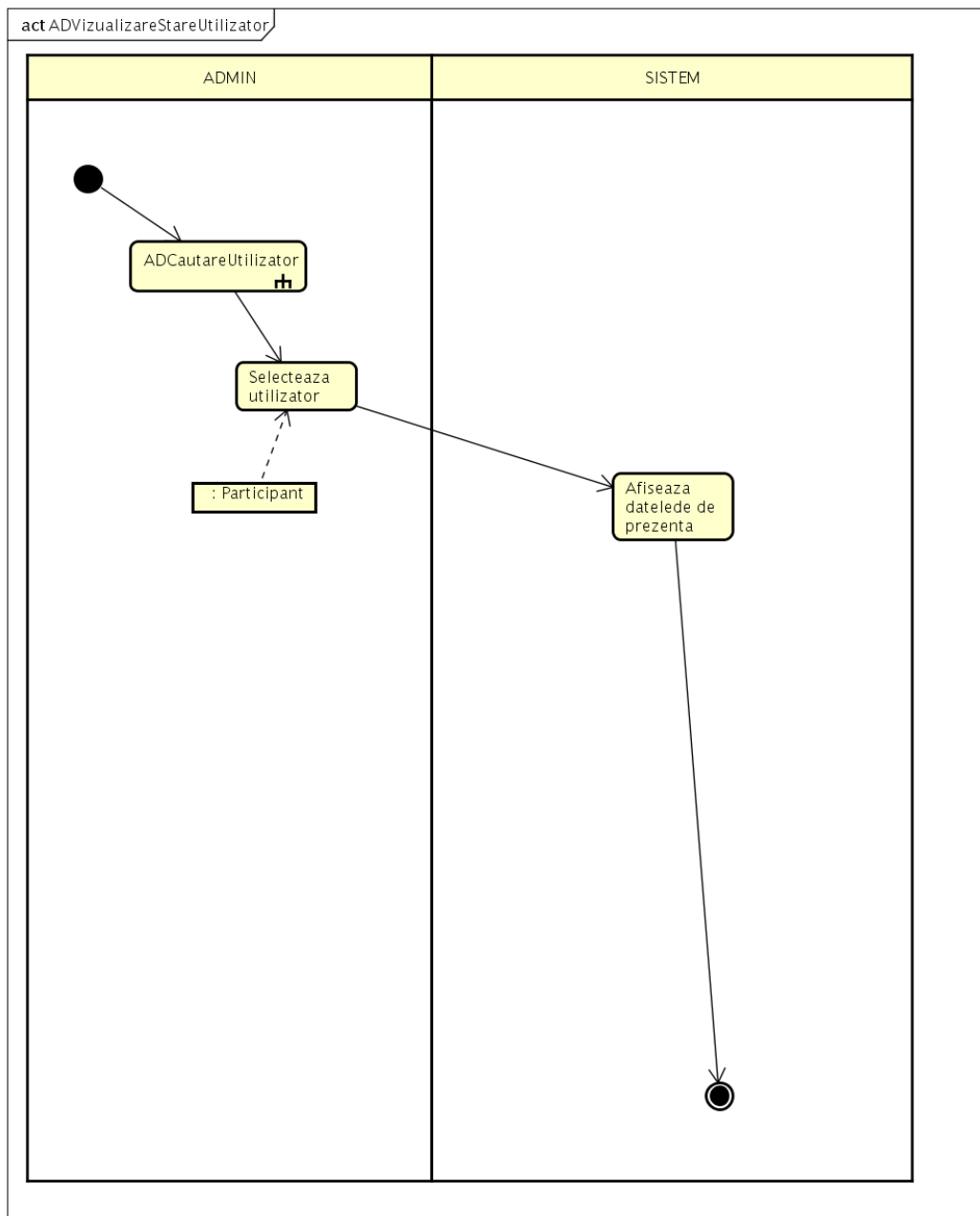
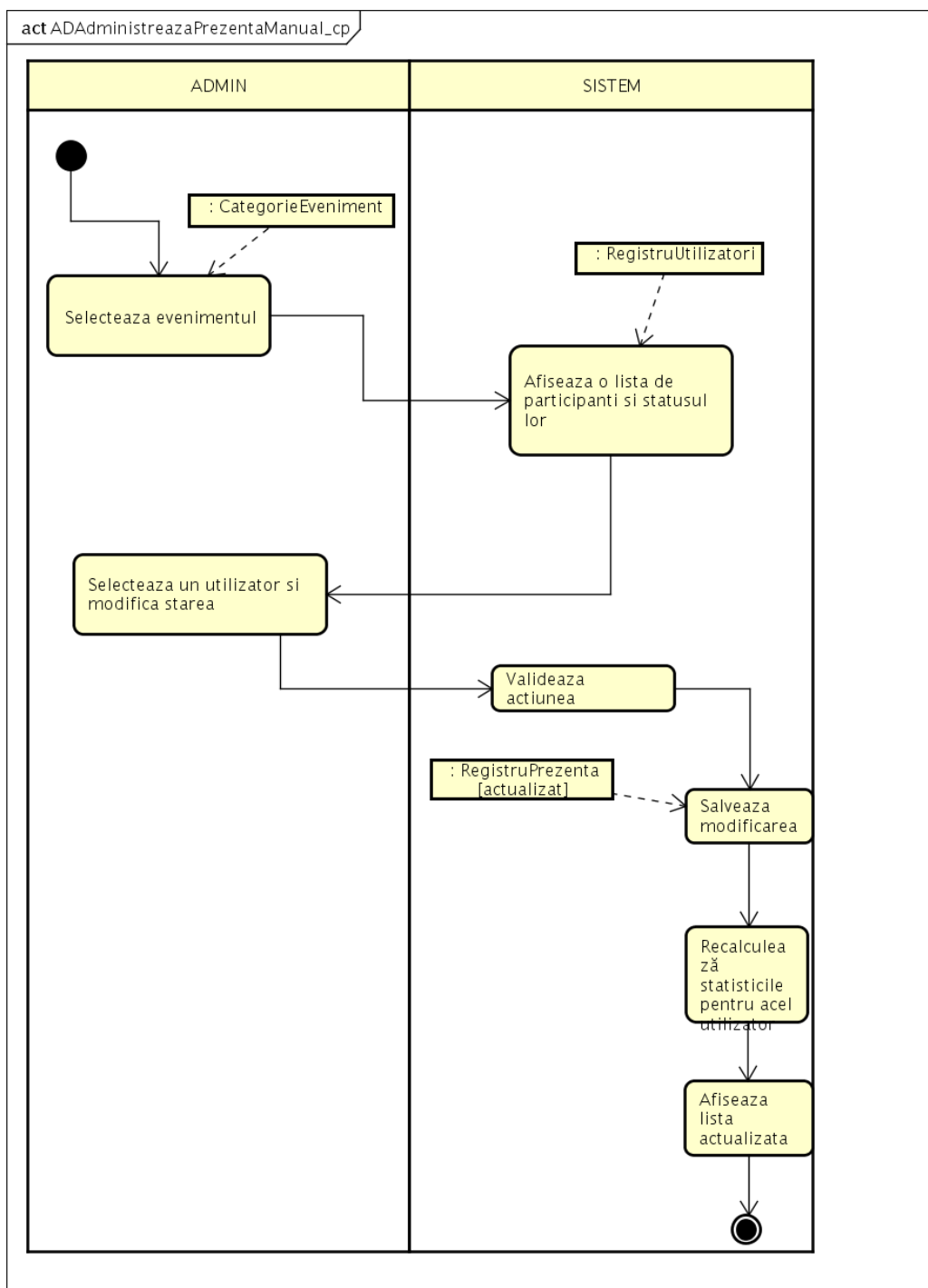


Figura 6.10: Diagrama de activitate — Căutare utilizator



powered by Astah

Figura 6.11: Diagrama de activitate — Vizualizare stare utilizator



powered by Astah

Figura 6.12: Diagrama de activitate — Administrare prezență manuală

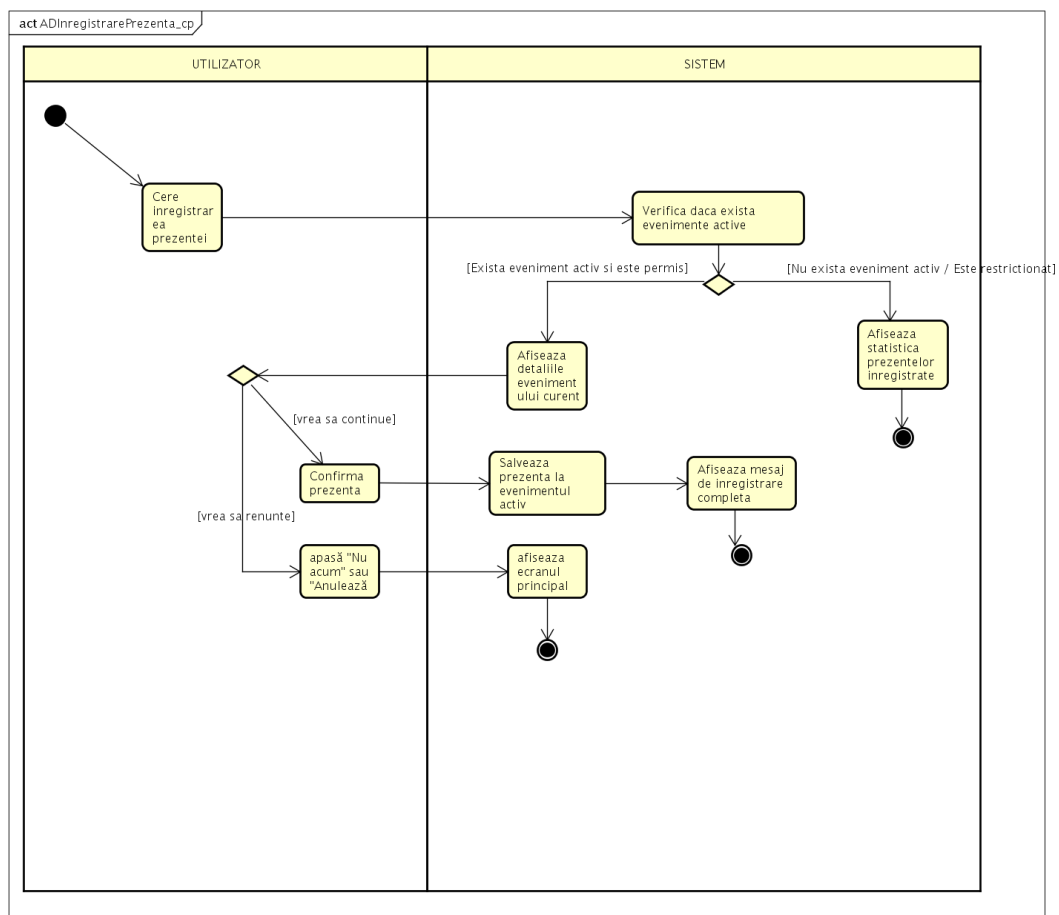
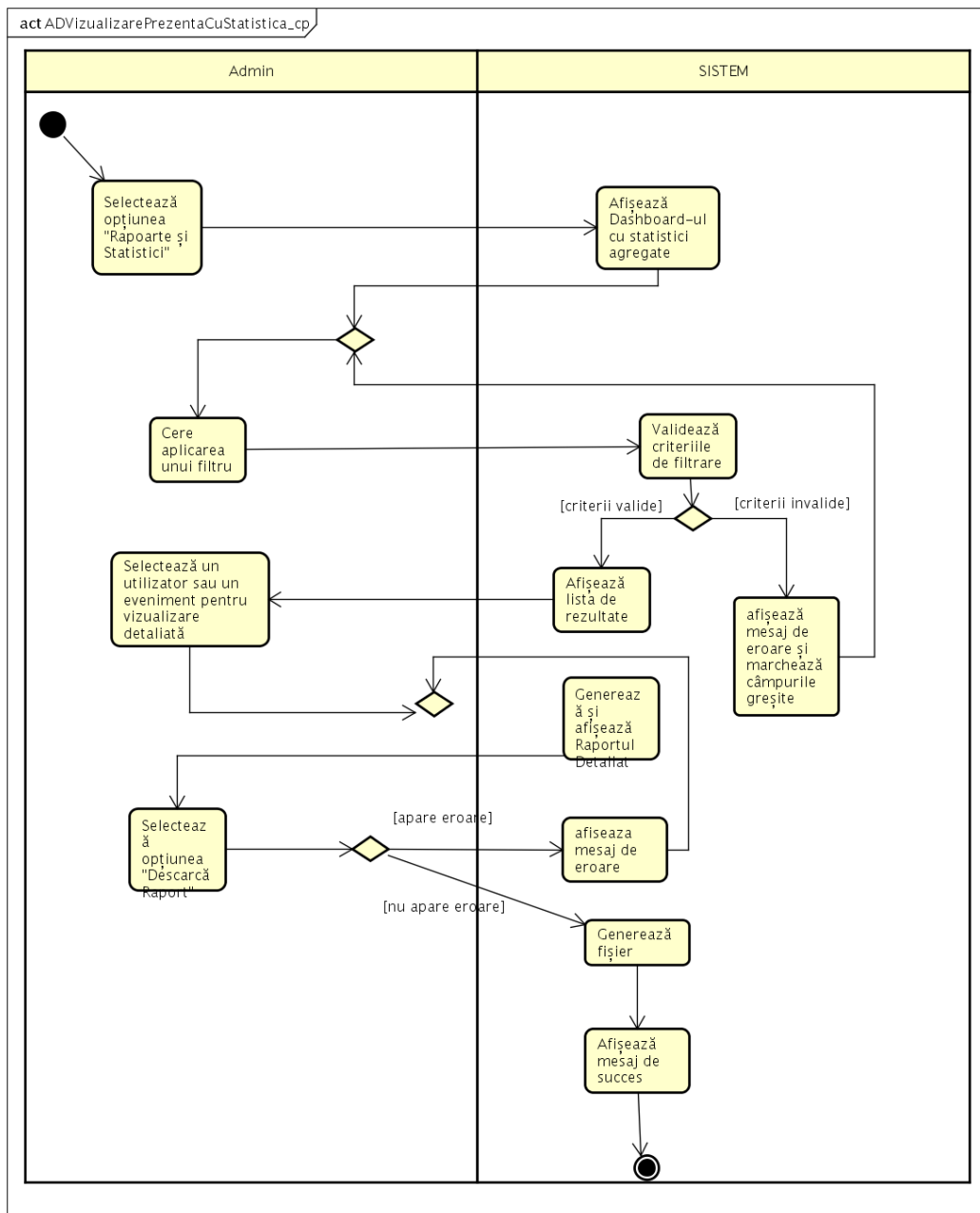


Figura 6.13: Diagrama de activitate — Înregistrare prezență



powered by Astah

Figura 6.14: Diagrama de activitate — Vizualizare prezență cu statistică

Referințe bibliografice

- [1] Andrew S. Tanenbaum. *Rețele de calculatoare*. Byblos, 4 edition, 2004.
- [2] Crenguța-Mădălina Puchianu. *Ingineria sistemelor software — note de curs*. Universitatea Ovidius din Constanța, Facultatea de Matematică și Informatică, 2025.
- [3] Mark Grand. *Patterns in Java: A Catalog of Reusable Design Patterns Illustrated with UML*, volume 1. Wiley, 2002.
- [4] Marten Deinum, Koen Serneels, Colin Yates, Seth Ladd, and Christophe Vanfleteren. *Pro Spring MVC: With Web Flow*. Apress, 2012.